



UNIVERSITÀ DEGLI STUDI DI NAPOLI

FEDERICO II

TESINA BASI DI DATI

Anno Accademico 2019/20

Prof. Ing. Vincenzo Moscato, Ph.D

Realizzata da Sara Auditano (N46004332)



Indice

1. Introduzione
 - 1.1.1 La storia del Coronavirus: tutte le tappe della Covid-19
 - 1.1.2 Specifiche
2. Creazione Tabella Master
3. Normalizzazione
4. Arricchimento dello schema
5. Reverse – Engineering, possibile schema concettuale
6. Analytics sui dati
7. Programmazione basi di dati
 - 7.1.1 Stored – procedure
 - 7.1.2 Trigger
 - 7.1.3 Viste

LA STORIA DEL CORONAVIRUS: TUTTE LE TAPPE DELLA COVID-19

Mentre a fine dicembre 2019 e inizio gennaio 2020 pensavamo ai buoni propositi per l'anno nuovo ed eravamo del tutto ignari dell'emergenza sanitaria che si sarebbe creata, un nuovo virus altamente contagioso e completamente sconosciuto al nostro sistema immunitario aveva iniziato a circolare in una regione remota del globo. Non avremmo mai pensato, all'epoca, che questo virus apparentemente così lontano avrebbe potuto diffondersi e causare tanti problemi a livello individuale e collettivo, per la salute, per i sistemi sanitari ed economici. Ma in poco più di due mesi lo scenario globale è cambiato radicalmente e noi abbiamo dovuto adattarci e far fronte alle nuove esigenze.

31 dicembre 2019: “polmoniti anomale”

Già a novembre – e forse anche a ottobre, secondo le ipotesi di uno studio italiano – il nuovo coronavirus **Sars-CoV-2** aveva iniziato a circolare, in Cina, in particolare a **Wuhan**, la città più popolata della parte orientale, perno per il commercio e gli scambi. All'inizio, però, non si sapeva si trattasse di un nuovo virus: ciò che inizia ad essere registrato è un certo numero di polmoniti anomale, dalle cause non ascrivibili ad altri patogeni.

La prima data ufficiale in cui inizia la storia del nuovo coronavirus è il **31 dicembre**, in le autorità sanitarie locali avevano dato notizia di questi casi insoliti. All'inizio di gennaio 2020 la città aveva riscontrato decine di casi e centinaia di persone erano sotto osservazione. Dalle prime indagini infatti, era emerso che i contagiati erano frequentatori assidui del mercato *Huanan Seafood Wholesale Market* a Wuhan, che è stato chiuso dal 1 gennaio 2020, di qui l'ipotesi che il contagio possa essere stato causato da qualche prodotto di origine animale venduto nel mercato.

21 gennaio: il virus si trasmette fra esseri umani

Il 21 gennaio le autorità sanitarie locali e l'Organizzazione mondiale della sanità annunciavano che il nuovo coronavirus, passato probabilmente dall'animale all'essere umano (un salto di specie, in gergo tecnico), si trasmette anche **da uomo a uomo**. Ma ancora gli esperti non sapevano (e tuttora l'argomento è discusso) quanto facilmente questo possa avvenire.

In Italia i casi erano pochissimi e tutti provenienti dalla Cina: a partire dal 29 gennaio c'erano **due turisti cinesi** di Wuhan contagiati, ricoverati allo **Spallanzani** – uno degli ospedali italiani che saranno protagonisti (loro malgrado) della vicenda del coronavirus.

C'era poi un ricercatore italiano positivo al virus e proveniente dalla Cina e un diciassettenne, rimasto bloccato a lungo a Wuhan a causa di sintomi simil-influenzali, non positivo al coronavirus ma ugualmente tenuto sotto osservazione e ricoverato allo Spallanzani. Tutte queste persone sono guarite e sono state dimesse nel mese di febbraio – per ultima, la paziente cinese della coppia malata, il 26 febbraio. I contagi fuori dalla Cina sono ancora molto circoscritti e limitati, con focolai per ogni paese di un manipolo di persone.

30 gennaio: l'Oms dichiara lo stato di emergenza globale

Alla fine di gennaio il rischio che l'epidemia si diffondesse passava da moderato a alto e il 27 gennaio l'Organizzazione mondiale della sanità scriveva che era *“molto alto per la Cina e alto a livello regionale e globale”*. Tanto che nella serata del 30 gennaio l'Oms dichiarava l’**“emergenza sanitaria pubblica di interesse internazionale”** e l'Italia bloccava i voli da e per la Cina, unica in Europa. Ma la situazione in Cina stava già migliorando: pochi giorni dopo, alla data dell'8 febbraio, l'Oms scriveva che i contagi in Cina si stavano **stabilizzando** ovvero che il numero di nuovi casi giornalieri sembrava andare **progressivamente calando**.

Febbraio: dare un nome alle cose

L'11 febbraio è arrivato il nome della nuova malattia causata dal coronavirus. Il nome, scelto dall'Oms, è **Covid-19**: *Co* e *vi* per indicare la famiglia dei coronavirus, *d* per indicare la malattia (*disease* in inglese) e infine 19 per sottolineare che sia stata scoperta nel 2019. Questo per quanto riguarda la malattia, mentre il virus cambia nome e non si chiama più 2019-nCoV, ma **Sars-CoV-2** perché il patogeno è parente del coronavirus responsabile della Sars (che però era molto più letale anche se meno contagiosa).

All'epidemia di Covid-19 si affianca quella dell'informazione, con notizie non sempre veritiere (molte sono fake news). Tanto che ai primi di febbraio proprio l'Oms parla per la prima volta di **infodemia**, termine nuovo con cui si indica il sovraccarico di aggiornamenti e news non sempre attendibili.

21 febbraio: primi casi in Italia

Venerdì 21 febbraio 2020 è una data centrale per la vicenda italiana legata al nuovo coronavirus. In questa data sono emersi diversi casi di coronavirus nel lodigiano, in Lombardia: si tratta di persone **non provenienti dalla Cina**, un nuovo focolaio di cui non si conosce ancora l'estensione.

Fra la fine di febbraio e i primi giorni di marzo 2020, dopo l'Italia, anche in altri stati (europei e non solo) vengono rilevare un numero crescente di casi e un'epidemia.

4, 8 e 9 marzo: le tre date chiave dei provvedimenti in Italia

Il contagio si è diffuso nel nostro paese, soprattutto nel nord, ma inizia anche in altre regioni. Per questo, mercoledì 4 marzo il governo ha dato il via libera alla chiusura di scuole e università in tutta Italia fino al 15 marzo. Alla data del 4, stando ai dati della Protezione civile i positivi sono circa 2.700 e già c'è qualche caso (decine o qualche unità) in tutte le regioni. Mentre domenica 8 marzo arriva il decreto che prevede l'isolamento della Lombardia, in assoluto la più colpita, e di altre 14 province, che diventano **"zona rossa"**.

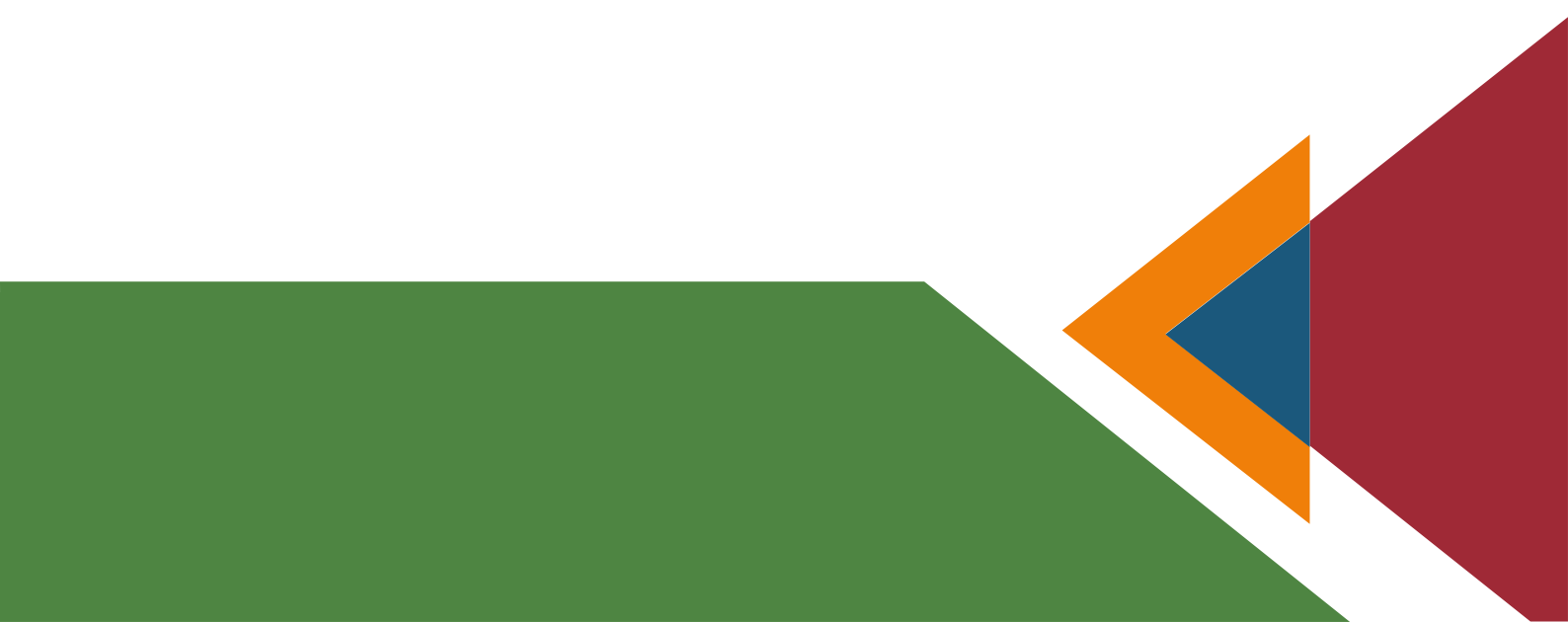


Anche se la **bozza ancora non ufficiale** del decreto era stata pubblicata da alcune testate già nella serata del 7.

E infine si arriva all'ultima data importante per l'Italia: quella di **lunedì 9 marzo**. In questa giornata, intorno alle 22, Conte annuncia in televisione di aver esteso a tutto il paese le misure già prese per la Lombardia e per le altre 14 province, tanto che tutta l'Italia diventerà *"zona protetta"*. Le nuove norme sono contenute nel **nuovo decreto Dpcm 9 marzo 2020**, entrato poi in vigore il 10 marzo. Di fatto la regola è contenuta nell'hashtag #iorestoacasa, si può uscire solo per comprovate ragioni di necessità come per fare la spesa, per esigenze lavorative, per l'acquisto di farmaci o per altri motivi di salute.

11 marzo: l'Oms dichiara la pandemia

Mentre l'Italia si sta muovendo – per prima in Europa, con il plauso dell'Organizzazione mondiale della sanità – per contenere il contagio, anche a livello globale sta succedendo qualcosa. L'11 marzo 2020 Tedros Adhanom Ghebreyesus, direttore generale dell'Oms, ha annunciato nel briefing da Ginevra sull'epidemia di coronavirus che Covid-19 *"può essere caratterizzato come una situazione pandemica"*. dichiarando la **pandemia**.



SPECIFICHE DI PROGETTO

Dalle specifiche dettate dal committente (Dato il DATASET, costituito dall'insieme dei file in formato CSV scaricabili da <https://github.com/pcm-dpc/COVID-19/tree/master/dati-province>), sono emerse le seguenti richieste:

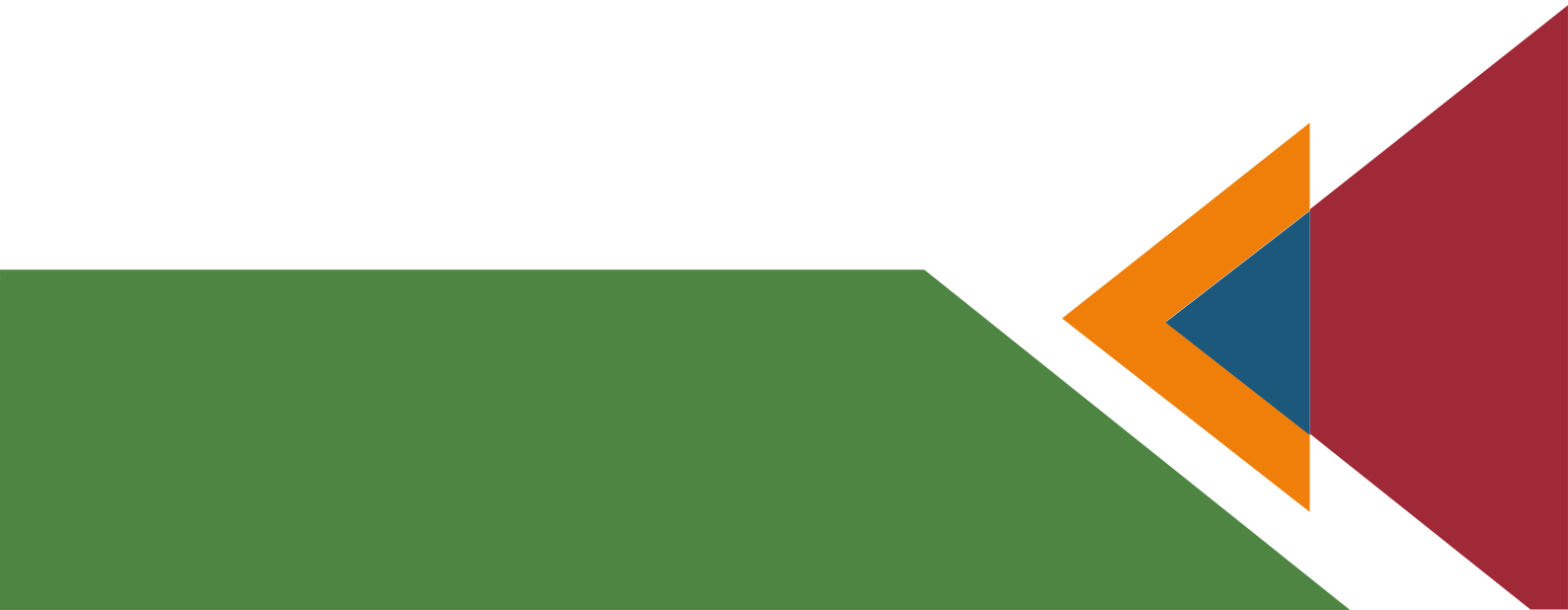
- 1) La creazione di una tabella master (con lo stesso schema dei file in formato CSV) ed il relativo popolamento con i dati del contagio per tutte le province italiane dal 25/02/2020 al 03/05/2020;
- 2) La verifica della 3NF per lo schema della tabella precedentemente istanziata e l'eventuale decomposizione con la definizione di tutti i vincoli;
- 3) L'arricchimento dello schema ottenuto con ulteriori informazioni utili all'analisi del fenomeno, ad esempio:
 - a. per ogni data il numero di morti, il numero di ricoveri in strutture ospedaliere, il numero di ricoveri in terapia intensiva, etc.
 - b. per ogni regione il numero di abitanti, la densità abitativa, superficie in km², numero di autostrade e strade statali, numero di aeroporti e stazioni, etc.;
 - c. per ogni provincia il numero di abitanti, la densità abitativa, il numero di scuole, alberghi/strutture recettive, il numero di ospedali, il numero di spostamenti intra e extra provincia, etc.
 - d. etc.
- 4) L'individuazione, attraverso un processo di reverse engineering, di un possibile schema concettuale E/R della base di dati;
- 5) La specifica in SQL di una serie di query utili all'analisi dell'andamento del contagio del COVID-19 (e.g., numero di contagi per provincia in una determinata finestra temporale, regione col il maggior numero di contagi per densità abitativa, etc.), con un'eventuale visualizzazione grafica dei risultati;



6) La specifica in PL/SQL di una serie di procedura/trigger che possano consentire un'analisi più flessibile ed eventuali aggiornamenti automatici della base di dati qualora vogliano essere importati dati successivi al 03/5/2020, ad esempio:

- a. STORED PROCEDURE aventi come parametri di ingresso le differenti variabili dell'analisi (es. giorno o finestra temporale di analisi, provincia o regione da analizzare, etc.);
- b. TRIGGER che all'atto di inserimenti nella tabella master esegue in automatico il popolamento delle tabelle dello schema normalizzato;

7) La definizione di viste sui dati ed indici che possono essere utili per migliorare i tempi di esecuzione delle query.



CREAZIONE TABELLA MASTER

Il primo passo per la creazione di una base di dati relazionale consiste nella definizione delle tabelle e dei vincoli di tipo intra-relazionale e inter-relazionale sulle tabelle stesse.

```
CREATE TABLE COVID(  
  data DATE,  
  stato CHAR(3),  
  codice_regione NUMBER(2),  
  denominazione_regione VARCHAR2(200),  
  codice_provincia NUMBER(3),  
  denominazione_provincia VARCHAR2(200),  
  sigla_provincia CHAR(2),  
  latitudine NUMBER,  
  longitudine NUMBER,  
  totale_casi INTEGER,  
  note_it CLOB,  
  note_en CLOB,  
  CONSTRAINT PK_COVID primary key(data,codice_provincia)  
);
```

L'istruzione di creazione di una tabella (**create table**) prevede la specifica del nome seguita dalla definizione dell'elenco dei *campi* o *attributi* della tabella stessa in termini di nome del campo e tipo di appartenenza. Tale istruzione avrà l'effetto di creare una tabella COVID (vuota) avente come colonne i nomi dei campi.

Il DBMS controllerà che le informazioni che si vanno a inserire siano poi compatibili con i tipi dei campi: ad esempio che il codice di una regione sia una stringa a dimensione fissa costituita da 3 caratteri.

Possiamo notare nella riga 14 del codice la presenza di un vincolo: vincolo di *chiave primaria* o *primary key*; tale vincolo assicura che i record della tabella stessa abbiano valori distinti e non nulli sul campo in questione.

Creata la tabella, possiamo inserire i dati ricavati dalla Protezione Civile in merito ai contagi nelle province italiane tramite l'istruzione DML “*insert*”.

Esempio:

```
INSERT INTO COVID VALUES ('25-Feb-2020', 'ITA', 15, 'Campania', 063, 'Napoli', 'NA', 40.83956555, 14.25084984, 0, NULL, NULL);
```

Notiamo, però, che le province autonome di Trieste e Trento vengono definite dallo stesso Codice_Regione ‘04’, c’è bisogno, quindi, di aggiornare la tabella per renderla più attendibile.

L’operazione DML utilizzata è l’**update**.

```
update COVID set codice_regione=21
where codice_regione=4 AND denominazione_regione='P.A. Bolzano';

update COVID set codice_regione=22
where codice_regione=4 AND denominazione_regione='P.A. Trento';
```

Tutti i record che verificano la condizione logica espressa dalla clausola **where** saranno soggetti all’aggiornamento del valore dei campi esplicitato dopo la clausola **set**.

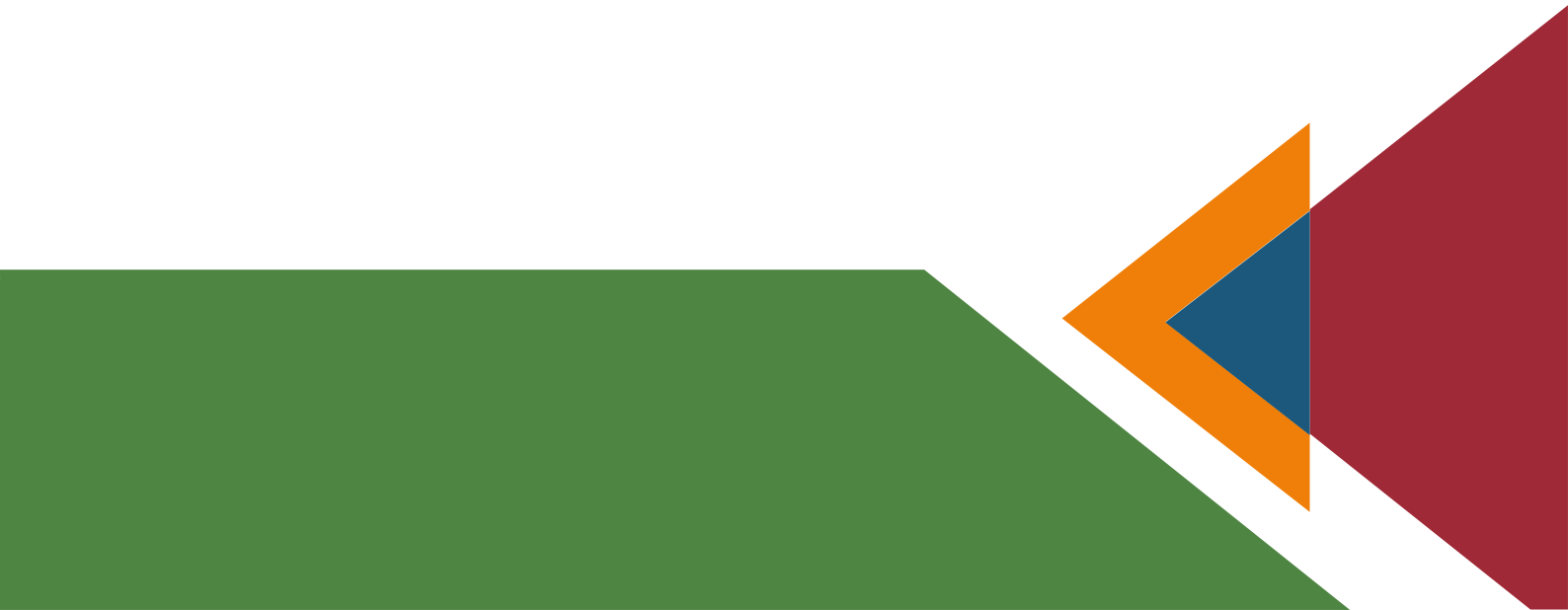
Un’ulteriore operazione DML molto importante è quella relativa alla cancellazione di record dalle tabelle (**delete**).



Notiamo che nella tabella COVID, alcune denominazioni delle province sono in fase di definizione/aggiornamento, come, ad esempio, il codice provincia 999 in data 19-APRILE-2020. Essendo la denominazione in fase di definizione, non sappiamo a quale provincia si riferisca il dato di 355 casi.

L'istruzione di **delete** provoca la cancellazione di tutti i record che soddisfano la condizione logica espressa dopo la clausola **where**, e possiamo utilizzarla per pulire la nostra tabella da quei dati superflui:

```
delete from COVID where denominazione_provincia='In fase di definizione/aggiornamento';
```



NORMALIZZAZIONE

Le forme normali sono state introdotte con l'intento di fornire un criterio di scelta tra i vari schemi relazionali che possono modellare una data realtà di interesse.

La normalizzazione è un procedimento volto all'eliminazione della ridondanza *informativa* e del rischio di *incoerenza* dal *database*.

Questo processo si fonda su un semplice criterio: se una *relazione* presenta più concetti tra loro indipendenti, la si decompone in relazioni più piccole, una per ogni concetto. Questo tipo di processo non è sempre applicabile in tutte le tabelle, dato che in alcuni casi potrebbe comportare una perdita d'informazioni.

Una decomposizione dovrebbe sempre soddisfare due proprietà:

- la *decomposizione senza perdita*, che garantisce la ricostruzione delle informazioni originarie
- la *conservazione delle dipendenze*, che garantisce il mantenimento dei vincoli di integrità originari.

Verifichiamo se la nostra Tabella Covid sia normalizzata:

```
CREATE TABLE COVID (data, stato, codice_regione, denominazione_regione,  
codice_provincia, denominazione_provincia , sigla_provincia , latitudine, longitudine,  
totale_casi, note_it, note_en)
```

1. Verifica 1NF

Per definizione, uno schema di relazione $R(X)$ è detto in prima forma normale se ogni attributo appartenente ad X è un attributo semplice, ovvero se per ogni attributo il relativo dominio è atomico.

Affinché una data relazione, quindi, sia in 1NF, non deve contenere nel suo schema attributi *multi valore* – ovvero quegli attributi i cui valori sono un insieme di valori – e attributi *strutturati* – ovvero attributi i cui valori sono ennuple di valori.

È possibile osservare che la tabella COVID è in 1NF in quanto ogni attributo è semplice.

2. Definire la chiave e le FD:

Per definizione, una chiave è un insieme di attributi utilizzato per identificare univocamente le tuple di una relazione, mentre una *dipendenza funzionale* è un particolare *vincolo di integrità* per il *modello relazionale* che descrive legami di tipo funzionale tra gli attributi di una relazione o tabella.

Per quanto riguarda la nostra tabella, la chiave sarà

- $K = \{\text{data}, \text{codice_provincia}\}$

Definiamo adesso le dipendenze funzionali:

- $\{\text{data}, \text{codice_provincia}\} \rightarrow \text{stato}, \text{codice_regione}, \text{denominazione_regione}, \text{denominazione_provincia}, \text{sigla_provincia}, \text{latitudine}, \text{longitudine}, \text{totale_casi}, \text{note_it}, \text{note_en}$
- $\text{codice_regione} \rightarrow \text{denominazione_regione}, \text{stato}$
- $\text{codice_provincia} \rightarrow \text{denominazione_regione}, \text{denominazione_provincia}, \text{sigla_provincia}, \text{latitudine}, \text{longitudine}, \text{codice_regione}, \text{denominazione_regione}, \text{stato}$

3. Verifica 2NF:

Per definizione, uno schema $R(X)$ di relazione è in seconda forma normale se:

- È in prima forma normale;
- Ogni attributo non primo di $R(X)$ è in dipendenza funzionale completa da ogni chiave di $R(X)$.

La tabella COVID, come si può notare dal punto 2, non è in 2NF (ci sono attributi che dipendono da parte della chiave: codice_provincia), bisogna, quindi, decomporla.

PROVINCE(codice_provincia, denominazione_regione, denominazione_provincia ,
sigla_provincia, latitudine, longitudine, codice_regione, denominazione_regione, stato)
COVID_PROVINCE(data, codice_provincia, totale_casi, note_it, note_en)

4. Verifica 3NF:

Per definizione, uno schema di relazione $R(X)$ è in terza forma normale se:

- È in 2NF;
- Ogni attributo non primo di $R(X)$ non dipende transitivamente da ogni chiave di $R(X)$.

Un attributo $A \subset X$ dipende transitivamente dall'insieme di attributi $Y \subset X$, se esiste un altro insieme di attributi $Z \subset X$ tale che:

- Y implica Z e Z non implica Y
- Z implica A e A non implica Z

Con A non appartenente a $Y \cup Z$

La nostra tabella COVID non è in 3NF in quanto si nota dai punti precedenti la presenza di dipendenze transitive degli attributi (stato e denominazione_regione dipendono transitivamente dalla chiave).

Normalizziamo decomponendo ancora una volta la tabella (gli attributi sottolineati sono chiavi esterne che deferenziano le tabelle in maiuscolo):

COVID_PROVINCE(data, codice_provincia:PROVINCE, totale_casi, note_it, note_en)
REGIONI(codice_regione, denominazione_regione, Stato)
PROVINCE(codice_provincia, denominazione_provincia, sigla_provincia, latitudine,
longitudine, codice_regione:REGIONI)

CREAZIONE TABELLA REGIONI:

```
CREATE TABLE REGIONI(  
  codice_regione NUMBER(2),  
  denominazione_regione VARCHAR2(200),  
  stato CHAR(3),  
  CONSTRAINT PK_REGIONI primary key(codice_regione)  
);  
  
INSERT INTO REGIONI  
  select distinct codice_regione, denominazione_regione, stato  
  from COVID  
  order by codice_regione;
```

CREAZIONE TABELLA PROVINCE:

```
CREATE TABLE PROVINCE(  
  codice_provincia NUMBER(3),  
  denominazione_provincia VARCHAR2(200),  
  sigla_provincia CHAR(2),  
  latitudine NUMBER,  
  longitudine NUMBER,  
  codice_regione NUMBER(2),  
  CONSTRAINT PK_PROVINCE primary key(codice_provincia),  
  CONSTRAINT FK_PROVINCE_REGIONI FOREIGN KEY(codice_regione) REFERENCES REGIONI(codice_regione)  
);
```

CREAZIONE TABELLA COVID_PROVINCE:

```
CREATE TABLE COVID_PROVINCE(  
  data DATE,  
  codice_provincia NUMBER(3),  
  totale_casi INTEGER,  
  annotazione_it CLOB,  
  annotazione_en CLOB,  
  CONSTRAINT PK_COVID_PROVINCE primary key(data, codice_provincia),  
  CONSTRAINT FK_COVID_PROVINCE_PROVINCE FOREIGN KEY(codice_provincia) REFERENCES PROVINCE(codice_provincia)  
);
```

ARRICCHIMENTO DELLO SCHEMA

Create e riempite le nostre tabelle, possiamo procedere con l'arricchimento dello schema ottenuto con ulteriori informazioni utili all'analisi del fenomeno.

Per procedere alla manipolazione degli schemi della nostra base di dati, il linguaggio SQL fornisce apposite primitive. In particolare, su schemi di basi di dati precedentemente creati risulta possibile modificare le definizioni di relazioni mediante l'utilizzo del comando DDL "ALTER TABLE".

Tramite il comando di ALTER TABLE è quindi possibile aggiungere o rimuovere vincoli ed attributi sullo schema della relazione.

Per riempire i nuovi campi creati, si utilizza il comando "UPDATE", una operazione che cambia i valori di uno o più attributi di tuple presenti in una relazione.

In caso di inserimento, il DBMS deve:

- Verificare che i valori inseriti siano appartenenti al dominio;
- Se si modifica una chiave primaria, analogamente all'INSERT, dee essere soddisfatto il relativo vincolo mentre analogamente alla DELETE vanno considerato le relative politiche di violazione dei vincoli;
- Se la modifica coinvolge una chiave esterna il DBMS deve verificare che il nuovo valore nella tabella referente sia presente nella tabella riferita, oppure sia NULL

Modifichiamo la tabella REGIONI:

```
ALTER TABLE REGIONI ADD numero_abitanti number;  
ALTER TABLE REGIONI ADD superficie number;  
ALTER TABLE REGIONI ADD abitanti_per_km_2 number;  
ALTER TABLE REGIONI ADD numero_comuni;  
ALTER TABLE REGIONI ADD numero_autostrade integer;  
ALTER TABLE REGIONI ADD numero_SS integer;  
ALTER TABLE REGIONI ADD numero_aeroporti integer;  
ALTER TABLE REGIONI ADD numero_stazioni integer;
```

Creiamo una nuova tabella COVID_REGIONI:

```
CREATE TABLE COVID_REGIONI(  
  Data date,  
  stato char(3),  
  codice_regione NUMBER(2),  
  denominazione_regione VARCHAR2(200),  
  Latitudine number,  
  Longitudine number,  
  Ricoverati_con_sintomi integer,  
  Terapia_intensiva integer,  
  Totale_ospedalizzati integer,  
  Isolamento_domiciliare integer,  
  Totale_positivi integer,  
  Variazione_totale_positivi integer,  
  Nuovi_positivi integer,  
  Dimessi_guariti integer,  
  Deceduti integer,  
  Totale_casi integer,  
  Tamponi integer,  
  casi_testati integer,  
  note_it clob,  
  note_en clob,  
  constraint PK_COVID_REGIONI primary key (Data, Codice_regione),  
  constraint FK_COVID_REGIONI foreign key (Codice_regione) references REGIONI(Codice_regione)  
);
```

CREAZIONE SCHEMA CONCETTUALE

La progettazione di una applicazione di basi di dati si suddivide in due fasi: la progettazione della base di dati vera e propria e la progettazione delle applicazioni di gestione delle informazioni presenti nella base di dati. Le due progettazioni hanno in comune una fase iniziale di raccolta dei requisiti descrittivi e operativi che dettagliano le esigenze dell'utente per il quale si costruisce l'applicazione.

La progettazione della base di dati procede secondo tre fasi distinte. La prima fase della progettazione delle basi di dati viene denominata progettazione concettuale. La traduzione del modello concettuale in un particolare modello logico dei dati è assicurata dalla fase di progettazione logica, in cui, usando anche i requisiti dinamici, si sceglie di implementare il modello concettuale nel modello logico del DBMS a disposizione. L'ultima tappa della progettazione consiste nella progettazione fisica nella quale si scelgono le particolari strutture di memorizzazione interna, gli indici di accesso ai dati e l'organizzazione dei file sui dischi.

Nel nostro caso si parlerà di un processo di "Reverse Engineering", quindi di progettazione inversa, in quanto esiste già la base dati relazionale da cui vogliamo ottenere uno schema concettuale.

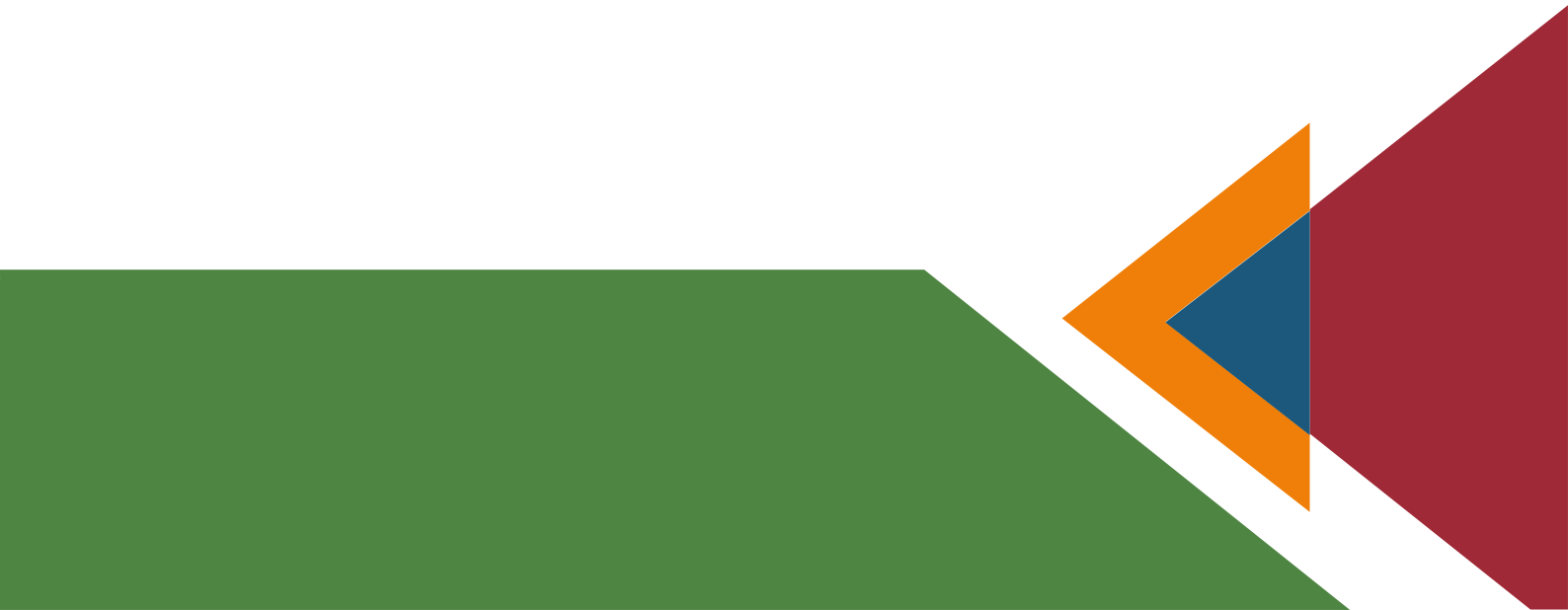
Il concetto chiave del modello ER è quello di entità. Per entità si intende un insieme di oggetti che possiedono le stesse proprietà. Nel nostro caso, le entità sono le tabelle REGIONI, PROVINCE, COVID_REGIONI E COVID_PROVINCE, rappresentate nel diagramma ER con un rettangolo contenente il proprio nome. Ogni entità ha un nome che la identifica univocamente nello schema, nel nostro caso le tabelle avevano nome al plurale (es. "REGIONI", "PROVINCE"..) mentre nello schema E/R i nomi sono declinati al singolare. Gli attributi sono individuati da linee terminate da cerchi (colorati per gli identificatori d'identità) con i relativi nomi e sono riferiti ad una singola entità. Notiamo nel nostro diagramma la sola presenza di attributi semplici, atomici, in quanto stiamo andando a schematizzare tabelle normalizzate.

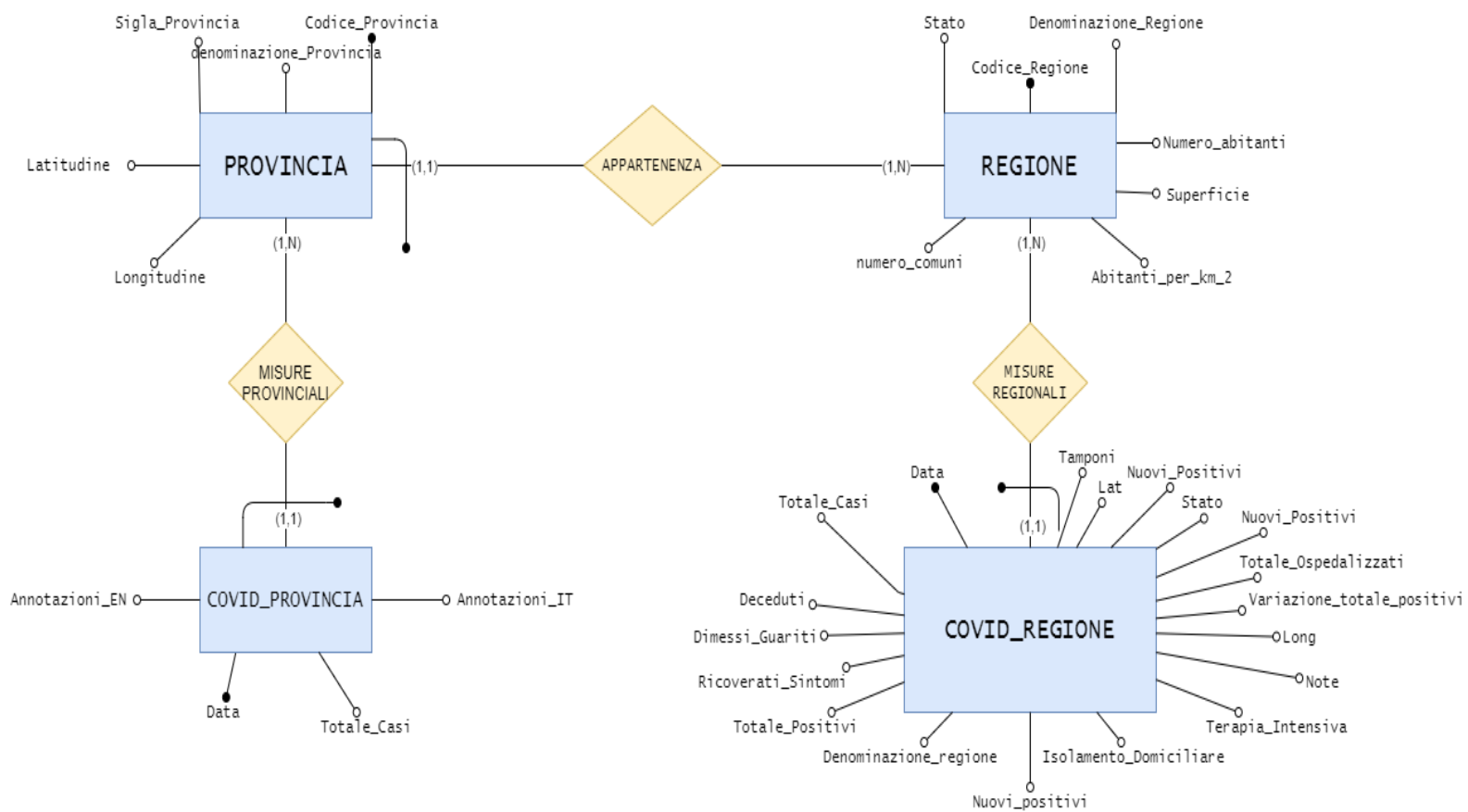


Nello schema notiamo anche dei rombi che collegano due entità, essi rappresentano le associazioni. Con il termine associazione si fa riferimento al legame concettuale fra due o più entità, rilevante in una applicazione di interesse. Dal punto di vista matematico, possiamo dire che ogni occorrenza di una associazione lega n occorrenze di entità. Le linee sono completate con la cardinalità dell'associazione: una coppia di valori associati ad ogni entità che specificano il numero minimo e massimo delle occorrenze di associazioni, cui ciascuna occorrenza di entità può partecipare. Con riferimento alle cardinalità massime, si può definire altresì il rapporto di cardinalità. Nel nostro schema troviamo due tipi di rapporti di cardinalità:

- “uno a uno”: con il quale si stabilisce che ad un'occorrenza di una entità corrisponde una ed una sola occorrenza dell'altra entità. Ad esempio, ogni provincia appartiene ad una e una sola regione.
- “uno a molti”: che fissa che ad una occorrenza di una entità sono associate più occorrenze dell'altra entità. Ad esempio, ad una regione appartengono più provincie.

Dopo la progettazione concettuale avviene la trasformazione del modello in quello logico relazionale, per quanto riguarda il nostro caso di “Reverse Engineering”, tutte le trasformazioni e semplificazioni sono già state adeguatamente svolte con la normalizzazione, quindi possiamo considerare il nostro schema finito.







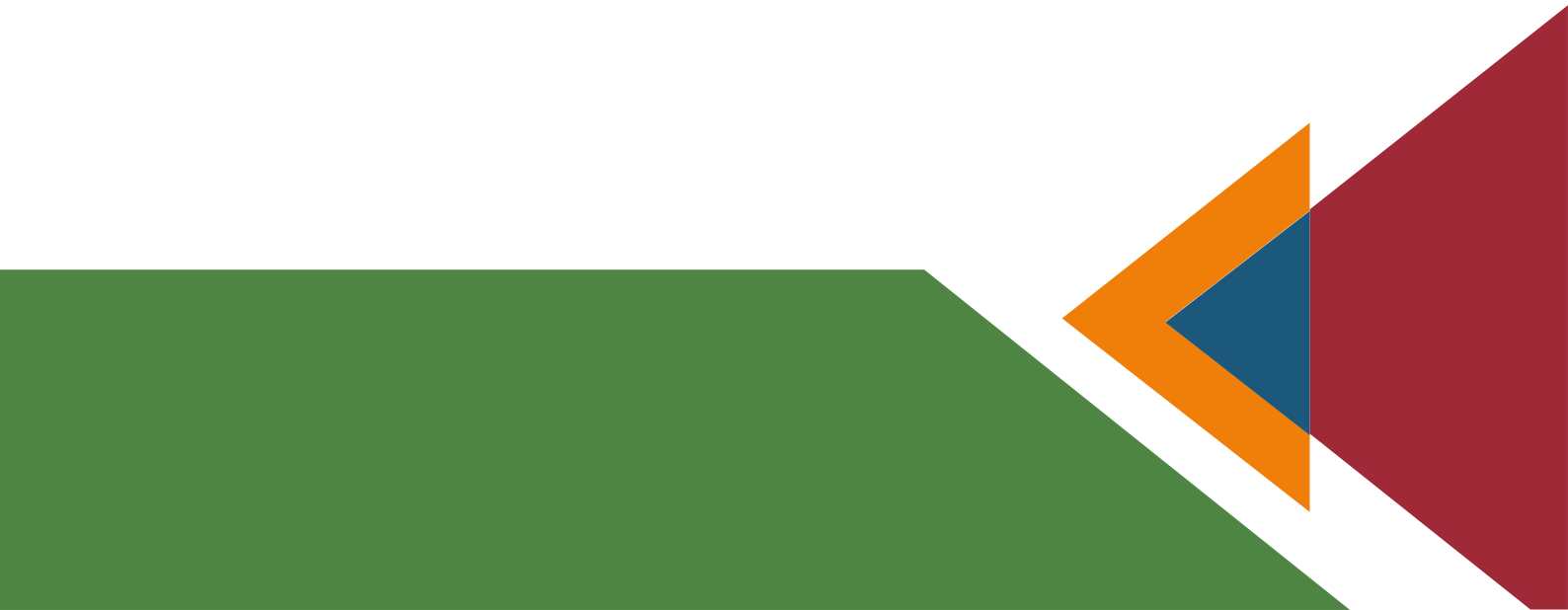
ANALYTICS SUI DATI

SQL è un linguaggio di definizione e di manipolazione dei dati. In quanto linguaggio di manipolazione, esso permette di selezionare dati di interesse dalla base e di aggiornarne il contenuto.

Le operazioni di interrogazione (Query di selezione, dette anche generalmente query) non alterano lo stato del database, nel senso che non inseriscono né aggiornano i valori dei dati, ma effettuano soltanto operazioni di lettura degli stessi. Esse hanno l'obiettivo di estrarre informazioni utili e sintetiche da tutte quelle memorizzate.

Ogni query è un programma (in linguaggio SQL) che può essere memorizzato e può costituire un "archivio di interrogazioni" da ripetere quando si voglia. In una base di dati relazionale ogni query equivale alla costruzione di una nuova tabella (relazione), tratta da quelle memorizzate (anche un singolo record è un caso particolare di relazione). La tabella viene esposta per la consultazione da parte dell'utente o può essere stampata come tabella o attraverso un report, ma non si aggiunge all'insieme delle tabelle che costituisce il database: lo stato del database resta invariato.

Query relativamente banali sono quelle cui si risponde consultando una singola tabella, query relativamente più complesse quelle che richiedono l'accesso a più tabelle. Nel complesso, esse si possono classificare in:

- selezione e proiezione, che operano su un'unica tabella;
 - join, che opera su più tabelle.
- 

QUERY N1 : TROVARE LA REGIONE CON DENSITA' MAGGIORE

```

select abitanti_per_km_2, denominazione_regione
from REGIONI
group by abitanti_per_km_2, denominazione_regione
having abitanti_per_km_2 >= ALL(select abitanti_per_km_2 from regione);

```

risultato query x

Tutte le righe recuperate: 1 in 0,01 secondi

ABITANTI_PER_KM_2	DENOMINAZIONE_REGIONE
753 P.A. Trento	

QUERY N2 : TROVARE PER CIASCUNA REGIONE LA QUANTITA' TOTALE DI TAMPONI ESEGUITI IN ITALIA

```

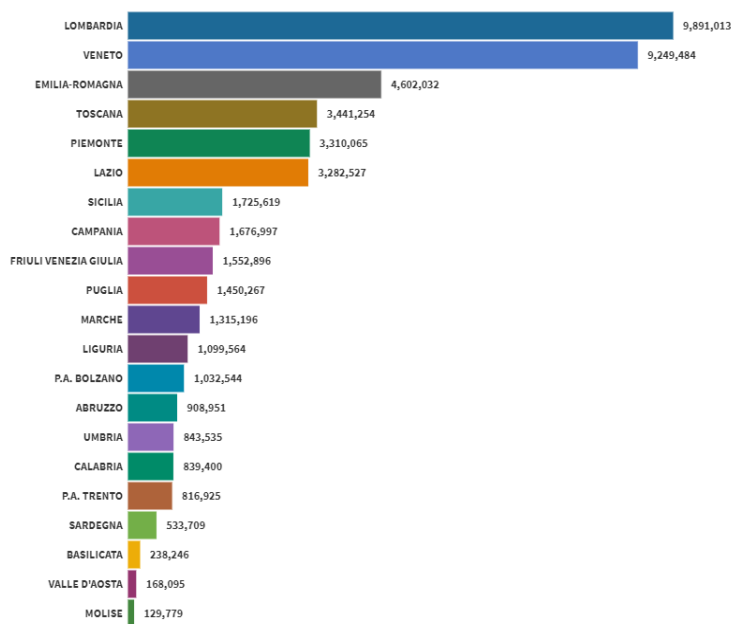
SELECT R.denominazione_regione, sum(tamponi) as Tamponi_Effettuati
from COVID_REGIONI C JOIN REGIONI R ON R.Codice_regione = C.Codice_regione
where R.Stato = 'ITA'
group by R.denominazione_regione;

```

Output script x Risultato query x

Tutte le righe recuperate: 21 in 0,002 secondi

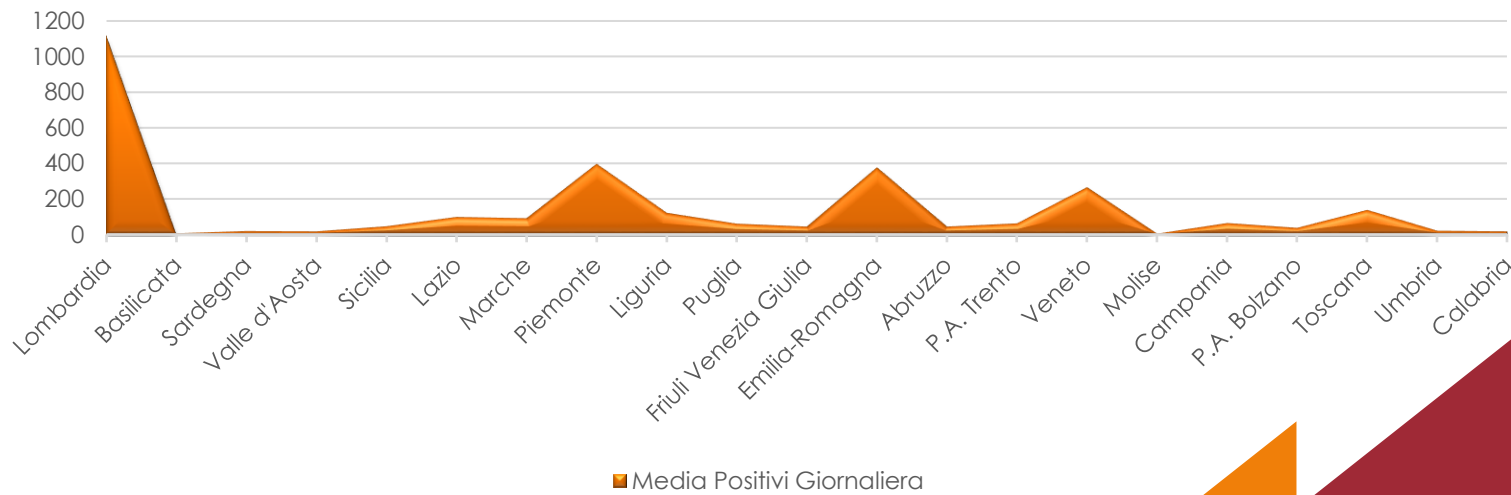
DENOMINAZIONE_REGIONE	TAMPONI_EFFETTUATI
1 Lombardia	9891013
2 Basilicata	238246
3 Sardegna	533709
4 Valle d'Aosta	168095
5 Sicilia	1725619
6 Lazio	3282527
7 Marche	1315196
8 Piemonte	3310065
9 Liguria	1099564
10 Puglia	1450267
11 Friuli Venezia Giulia	1552896
12 Emilia-Romagna	4602032
13 Abruzzo	908951
14 P.A. Trento	816925
15 Veneto	9249484
16 Molise	129779
17 Campania	1676597
18 P.A. Bolzano	1032544
19 Toscana	3441254
20 Umbria	843535
21 Calabria	839400



QUERY N3: TROVARE PER OGNI REGIONE LA QUANTITA' MEDIA DI NUOVI INFETTI

```
CREATE INDEX NOME ON COVID_REGIONI(denominazione_regioni);
select r.denominazione_regioni, avg(nuovi_positivi) as media_positivi_giornaliera
from COVID_REGIONI C JOIN REGIONI R ON R.Codice_regioni = C.Codice_regioni
where R.Stato = 'ITA'
group by R.denominazione_regioni;
```

DENOMINAZIONE_REGIONI	MEDIA_POSITIVI_GIORNALIERA
1 Lombardia	1121,101449275362318840579710144927536232
2 Basilicata	5,59420289855072463768115942028985507246
3 Sardegna	19,11594202898550724637681159420289855072
4 Valle d'Aosta	16,55072463768115942028985507246376811594
5 Sicilia	46,95652173913043478260869565217391304348
6 Lazio	98,63768115942028985507246376811594202899
7 Marche	91,57971014492753623188405797101449275362
8 Piemonte	397,492753623188405797101449275362318841
9 Liguria	121,144927536231884057971014492753623188
0 Puglia	60,05797101449275362318840579710144927536
1 Friuli Venezia Giulia	44,52173913043478260869565217391304347826
2 Emilia-Romagna	376,782608695652173913043478260869565217
3 Abruzzo	43,42028985507246376811594202898550724638
4 P.A. Trento	61,55072463768115942028985507246376811594
5 Veneto	265
6 Molise	4,36231884057971014492753623188405797101
7 Campania	64,98550724637681159420289855072463768116
8 P.A. Bolzano	36,75362318840579710144927536231884057971
9 Toscana	138,594202898550724637681159420289855072
0 Umbria	20,20289855072463768115942028985507246377
1 Calabria	16,14492753623188405797101449275362318841

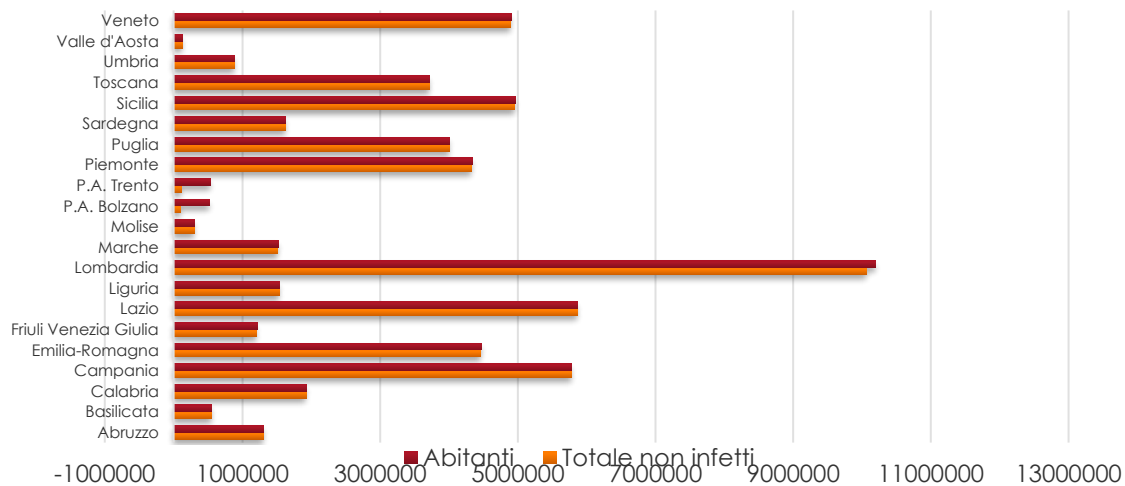


QUERY N4: TROVARE IN ORDINE ALFABETICO IL NUMERO DI ITALIANI PER REGIONE CHE NON HANNO CONTRATTO IL VIRUS AL 3 MAGGIO

```
SELECT R.denominazione_regione, R.NUMERO_ABITANTI - C.TOTALE_POSITIVI AS ABITANTI_NON_CONTAGIATI
FROM COVID_REGIONI C JOIN REGIONI R ON R.Codice_regione = C.codice_regione
WHERE R.Stato = 'ITA' AND C.DATA = '03-MAG-20'
GROUP BY R.denominazione_regione, C.Totale_positivi, r.numero_abitanti
ORDER BY R.denominazione_regione;
```

DENOMINAZIONE_REGIONE	ABITANTI_NON_CONTAGIATI
1 Abruzzo	1303502
2 Basilicata	556740
3 Calabria	1923599
4 Campania	5783135
5 Emilia-Romagna	4458073
6 Friuli Venezia Giulia	1210270
7 Lazio	5861159
8 Liguria	1535576
9 Lombardia	10067043
10 Marche	1515202
11 Molise	302084
12 P.A. Bolzano	106286
13 P.A. Trento	116170
14 Piemonte	4325737
15 Puglia	4005341
16 Sardegna	1629785
17 Sicilia	4550514
18 Toscana	3717401
19 Umbria	880102
20 Valle d'Aosta	125352
21 Veneto	4500405

Comparazione totale non infetti e abitanti per regione



QUERY N5: TROVARE IN ORDINE ALFABETICO LA PERCENTUALE DI INFETTI PER REGIONE ITALIANA

```
SELECT R.denominazione_regione, (100*(C.totale_positivi)/R.numero_abitanti) AS PERCENTUALE_CONTAGIATI
FROM COVID_REGIONI C JOIN REGIONI R ON R.Codice_regione = C.codice_regione
WHERE R.Stato = 'ITA' AND C.DATUM = '03-MAG-20'
GROUP BY R.denominazione_regione, C.totale_positivi, R.numero_abitanti
ORDER BY R.denominazione_regione;
```

DENOMINAZIONE_REGIONE	PERCENTUALE_CONTAGIATI
1 Abruzzo	0,1430573531326343843096410547033551064801
2 Basilicata	0,0348335709437743071850026466331737692438
3 Calabria	0,0364731976551163011813263462740446438174
4 Campania	0,0471140546430686907028374860716494917524
5 Emilia-Romagna	0,20247954050016140160165905624163050598724
6 Friuli Venezia Giulia	0,0897340750909529938077709543924705928971
7 Lazio	0,074758624264006884538934486101299385019
8 Liguria	0,2301171582118646095770142055708959793977
9 Lombardia	0,3654603453355805030676558884529278781437
10 Marche	0,2106164383561643935616438356164383561644
11 Molise	0,0598813300464823912791755578714042313864
12 P.A. Bolzano	0,621780067507550186534020252265055603061
13 P.A. Trento	1,06202679339448251133311190032107786777
14 Piemonte	0,3602084593014885936060441680343209236705
15 Puglia	0,0737221003638453946721898781926284865148
16 Sardegna	0,0422576502293198174273248147471226158773
17 Sicilia	0,04447704344557174805279180766373960228227
18 Toscana	0,1431208127156180318255774191460082106433
19 Umbria	0,0207807218344059026338060968890756970754
20 Valle d'Aosta	0,0868518975844414785539557453725468322562
21 Veneto	0,140725350996163763747728893185082066889



- Abruzzo
- Basilicata
- Calabria
- Campania
- Emilia-Romagna
- Friuli Venezia Giulia
- Lazio
- Liguria
- Lombardia
- Marche
- Molise
- P.A. Bolzano
- P.A. Trento
- Piemonte
- Puglia
- Sardegna
- Sicilia
- Toscana
- Umbria
- Valle d'Aosta
- Veneto

QUERY N6: TROVARE IN ORDINE ALFABETICO LE PROVINCE (CODICE, NOME, SIGLA) E LE RISPETTIVE REGIONI (CODICE, NOME, STATO) ITALIANE CON UN TOTALE POSITIVI MAGGIORE > 0 L'ULTIMA SETTIMANA DI MARZO

```

select distinct R.denominazione_regione, R.Codice_regione, R.Stato,
P.codice_provincia, P.denominazione_provincia, P.sigla_provincia
from (PROVINCE P join COVID C on C.Codice_provincia = P.Codice_Provincia) join REGIONI R on R.Codice_regione = C.Codice_regione
where Data > '23-Mar-20' and Data < '31-Mar-20' and totale_casi > 0
group by R.denominazione_regione, R.Codice_regione, R.Stato,
P.codice_provincia, P.denominazione_provincia, P.sigla_provincia;

```

DENOMINAZIONE_REGIONE	CODICE_REGIONE	STATO	CODICE_PROVINCIA	DENOMINAZIONE_PROVINCIA	SIGLA_PROVINCIA
1 Piemonte	1 ITA	4 Cuneo	CN		
2 Valle d'Aosta	2 ITA	7 Aosta	AO		
3 Friuli Venezia Giulia	6 ITA	32 Trieste	TS		
4 Liguria	7 ITA	8 Imperia	IM		
5 Emilia-Romagna	8 ITA	37 Bologna	BO		
6 Toscana	9 ITA	47 Pistoia	PT		
7 Toscana	9 ITA	100 Prato	PO		
8 Puglia	16 ITA	71 Foggia	FG		
9 Puglia	16 ITA	75 Lecce	LE		
10 Calabria	18 ITA	80 Reggio di Calabria	RC		
11 Piemonte	1 ITA	1 Torino	TO		
12 Veneto	5 ITA	25 Belluno	BL		
13 Emilia-Romagna	8 ITA	39 Ravenna	RA		
14 Emilia-Romagna	8 ITA	35 Reggio nell'Emilia	RE		
15 Toscana	9 ITA	52 Siena	SI		
16 Toscana	9 ITA	51 Arezzo	AR		
17 Toscana	9 ITA	53 Grosseto	GR		
18 Toscana	9 ITA				

QUERY N7: TROVARE TUTTE LE PROVINCE CHE APPARTENGONO A REGIONI IL CUI NOME INIZIA PER CONSONANTE E CHE IL GIORNO 7 MARZO AVEVANO TOTALE CASI PARI A ZERO

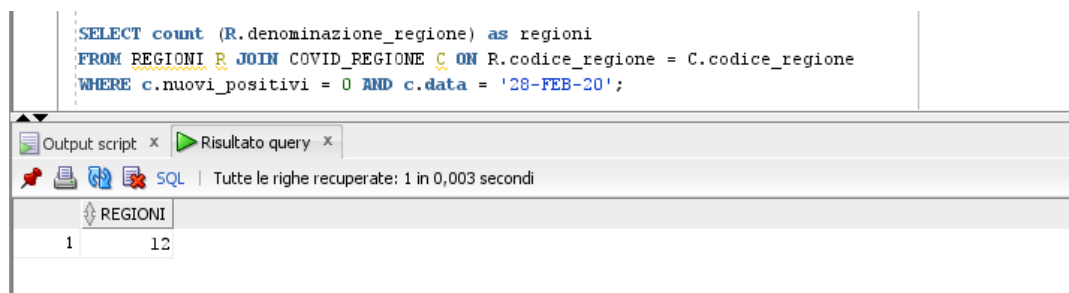
```

SELECT P.denominazione_provincia, C.denominazione_regione
FROM PROVINCE P JOIN COVID C ON C.denominazione_provincia = P.denominazione_provincia
WHERE C.Data = '07-Mar-20' and C.TOTALE_CASI = 0
GROUP BY P.denominazione_provincia, C.denominazione_regione
INTERSECT
SELECT P.denominazione_provincia, r.denominazione_regione
FROM PROVINCE P JOIN REGIONI R ON R.CODICE_REGIONE = P.CODICE_REGIONE
WHERE R.denominazione_regione <> 'AV' OR R.denominazione_regione <> 'EV' OR R.denominazione_regione <> 'IV' OR R.denominazione_regione <> 'OV' OR R.denominazione_regione = 'UV'
GROUP BY P.denominazione_provincia, r.denominazione_regione;

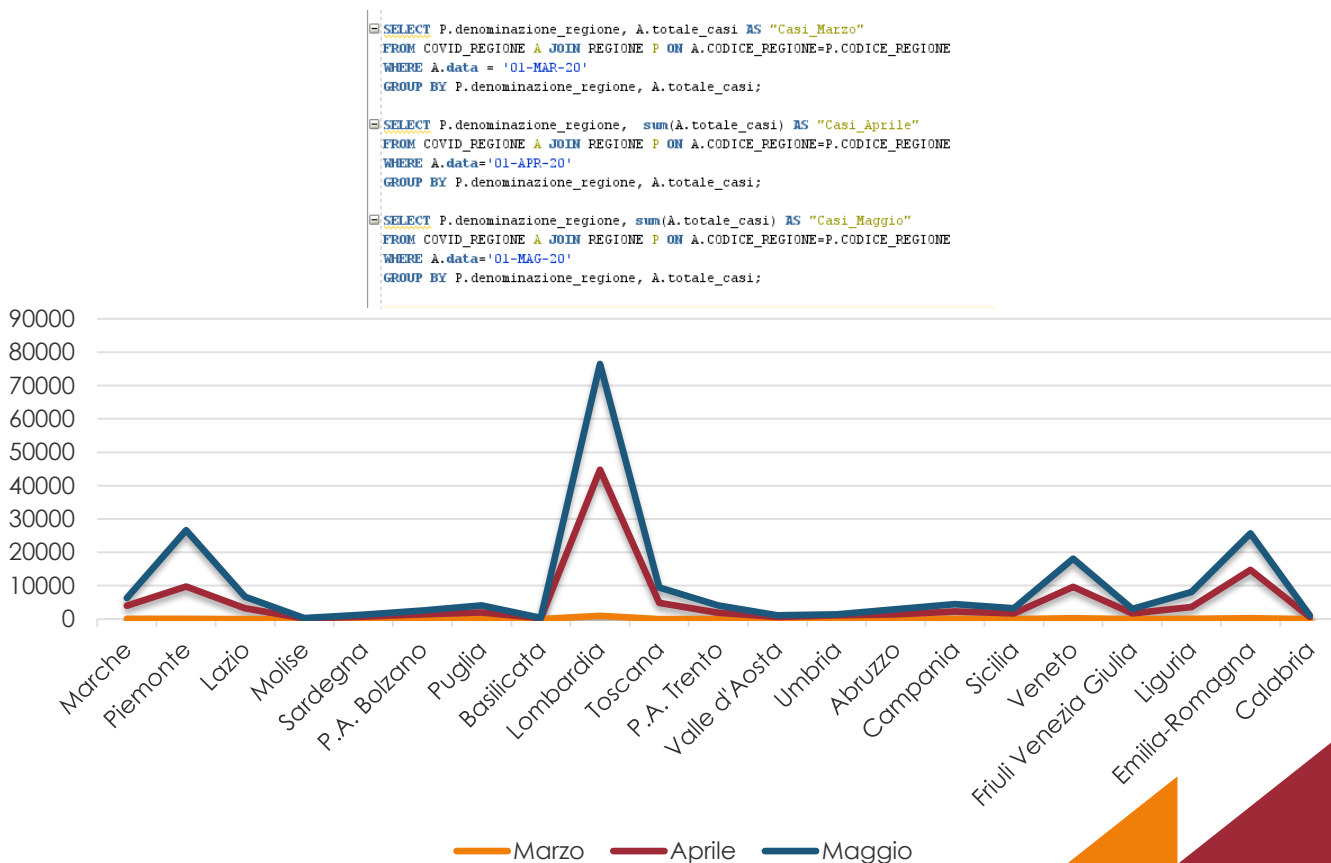
```

DENOMINAZIONE_PROVINCIA	DENOMINAZIONE_REGIONE
1 Ascoli Piceno	Marche
2 Arellino	Campania
3 Benevento	Campania
4 Caltanissetta	Sicilia
5 Caserta	Campania
6 Crotone	Calabria
7 Enna	Sicilia
8 Isernia	Molise
9 Oristano	Sardegna
10 Pieti	Lazio
11 Salerno	Campania
12 Sassari	Sardegna
13 Sud Sardegna	Sardegna
14 Trapani	Sicilia
15 Vibo Valentia	Calabria

QUERY N9: TROVARE IL NUMERO DI REGIONI CHE IL 28 FEBBRAIO AVEVANO ZERO NUOVI CASI



QUERY N10: ANDAMENTO POSITIVI OGNI PRIMO DEL MESE



QUERY N11: REGIONE IN CUI E' PRESENTE IL MAGGIOR NUMERO DI INFETTI

```
SELECT R.DENOMINAZIONE_REGIONE, SUM(C.TOTALE_CASI) as INFETTI_TOTALI
FROM REGIONE R JOIN COVID_REGIONE C ON R.CODICE_REGIONE = C.CODICE_REGIONE
GROUP BY R.DENOMINAZIONE_REGIONE
HAVING SUM(C.totale_casi) >= ALL ( SELECT SUM(C.totale_casi) as totale_casi
FROM REGIONE R JOIN COVID_REGIONE C ON R.CODICE_REGIONE = C.CODICE_REGIONE
GROUP BY R.DENOMINAZIONE_REGIONE
);
```

Output script x Risultato query x

Tutte le righe recuperate: 1 in 0,014 secondi

DENOMINAZIONE_REGIONE	INFETTI_TOTALI
1 Lombardia	2640680

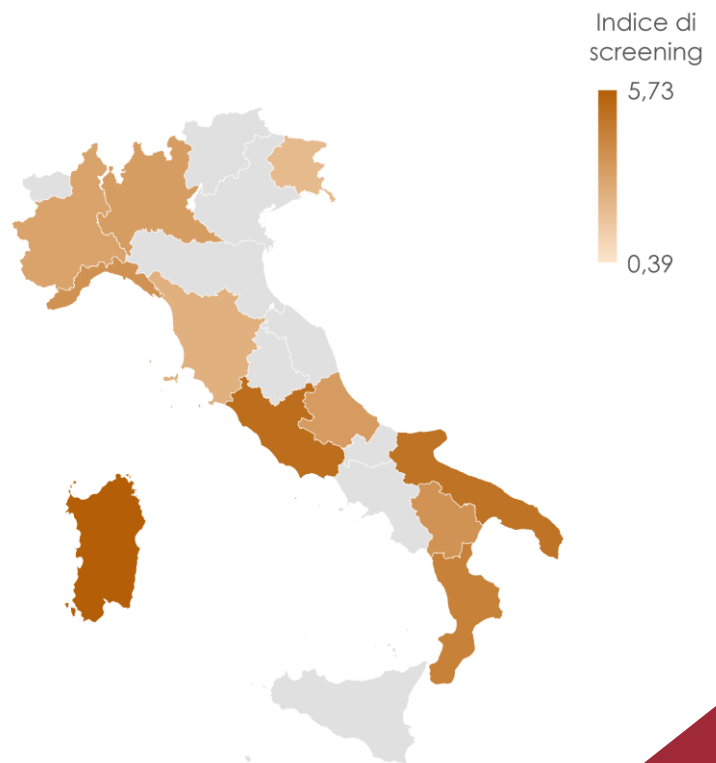
QUERY N12: INDICE DI SCREENING PER OGNI REGIONE

```
SELECT R.DENOMINAZIONE_REGIONE, R.NUMERO_ABITANTI, SUM(C.CASI_TESTATI) AS TAMPONI_EFFETTUATI,
R.NUMERO_ABITANTI/(SUM(C.CASI_TESTATI)) AS INDICE_DI_SCREENING
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
GROUP BY R.DENOMINAZIONE_REGIONE, R.NUMERO_ABITANTI;
```

Risultato query x

Tutte le righe recuperate: 21 in 0,064 secondi

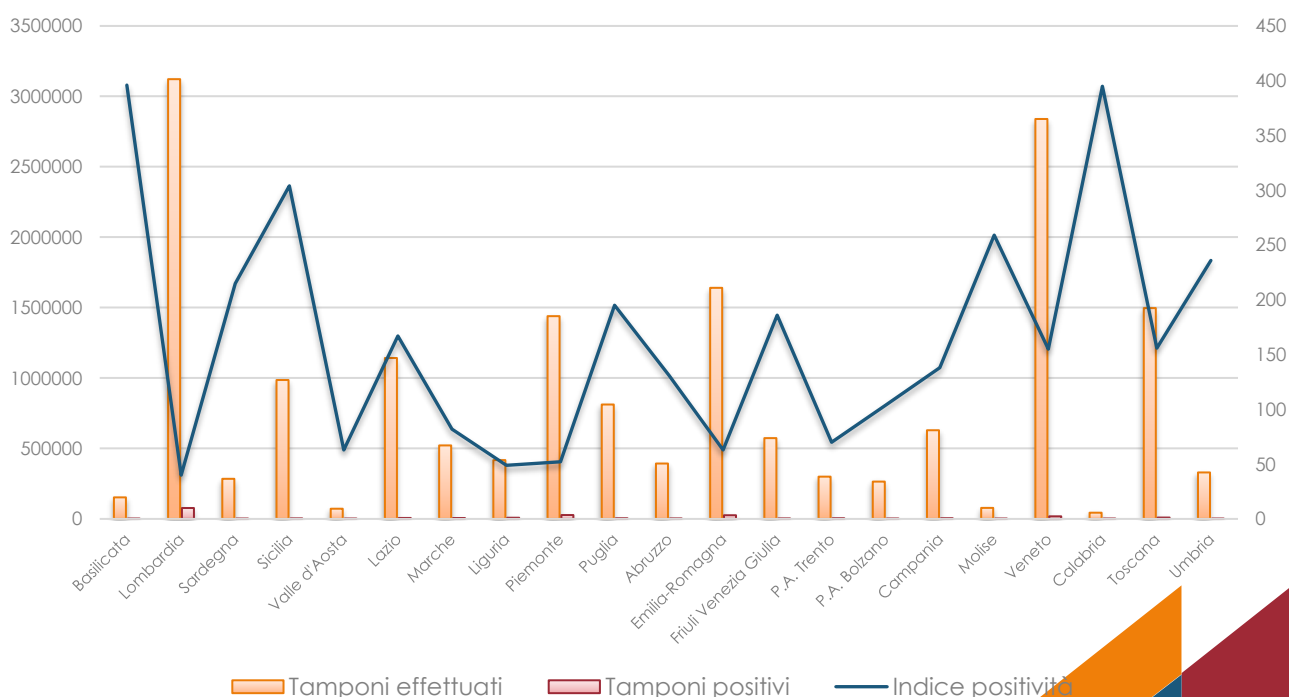
DENOMINAZIONE_REGIONE	NUMERO_ABITANTI	TAMPONI_EFFETTUATI	INDICE_DI_SCREENING
1 Friuli Venezia Giulia	1211357	573623	2,1117650442886704333682575489476537726
2 Basilicata	556934	152871	3,64316318987904834795350328054372640985
3 Piemonte	4341375	1439530	3,01582808277701749876695866011823303439
4 Toscana	3722729	1456786	2,48714846344059958177054034444469683709
5 P.A. Trento	117417	299982	0,3914134840090885453127187631257875472528
6 Calabria	1924701	440274	4,37159814115755188814238406083484375639
7 Lazio	5865544	1142459	5,13413960588520025663940675332769053419
8 Liguria	1543127	417509	3,69603289988958321856534829189310889107
9 Puglia	4008296	812177	4,93524833604374415921652546181435820024
10 Sardegna	1630474	284661	5,72777444047481038849719490903214700995
11 Lombardia	10103969	3122163	3,23620803910622219275547112690785202438
12 Abruzzo	1305770	393156	3,32125161513485915107489139171219566788
13 Valle d'Aosta	125501	72318	1,73540474017533670732044580878895987168
14 Marche	1518400	521736	2,91028412037143689520803839489703604888
15 Molise	302265	78163	3,86711103719151004951191740337499840078
16 Veneto	4907704	2839418	1,72841899290629276844761849083157182211
17 P.A. Bolzano	106951	264866	0,403792861295900568589399225799219227836
18 Campania	5785861	620533	9,32401822304373820570380624398702405835
19 Emilia-Romagna	4467118	1640554	2,72293261910305908857617609661126668186
20 Sicilia	4968410	986136	5,03826044277868367040651593694987303982
21 Umbria	880285	329835	2,66886473539800203131868964785423014538



QUERY N12: INDICE DI POSITIVITA' TAMPONI PER REGIONE

```
SELECT R.DENOMINAZIONE_REGIONE, SUM(C.CASI_TESTATI) AS TAMPONI_EFFETTUATI, SUM(C.NUOVI_POSITIVI) AS DI_CUI_POSITIVI,
SUM(C.CASI_TESTATI)/SUM(C.NUOVI_POSITIVI) AS INDICE_POSITIVITA
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
GROUP BY R.DENOMINAZIONE_REGIONE;
```

DENOMINAZIONE_REGIONE	TAMPONI_EFFETTUATI	DI_CUI_POSITIVI	INDICE_POSITIVITA
1 Basilicata	152871	386	396,038860103626943005181347150259067358
2 Lombardia	3122163	77356	40,36096747505041625730389368633331609701
3 Sardegna	284661	1319	215,815769522365420354814253222137983321
4 Sicilia	986136	3240	304,362962962962962962962962962962963
5 Valle d'Aosta	72318	1142	63,32574430823117338003502626970227670753
6 Lazio	1142459	6806	167,860564208051719071407581545694975022
7 Marche	521736	6319	82,56622883367621459091628422218705491375
8 Liguria	417509	8359	49,94724248312118674482593611676037803565
9 Piemonte	1439530	27427	52,48587158639297043059758632004958617421
10 Puglia	812177	4144	195,988658301158301158301158301158301158
11 Abruzzo	393156	2996	131,226969292389853137516688918558077437
12 Emilia-Romagna	1640554	25998	63,10308485268097545965074236479729209939
13 Friuli Venezia Giulia	573623	3072	186,726236979166666666666666666666667
14 P.A. Trento	2999802	4247	70,63385919472568872145043560160113020956
15 P.A. Bolzano	264866	2536	104,442429022082010927444794952681388013
16 Campania	620533	4484	138,388269402319357716324710080285459411
17 Molise	78163	301	259,67774086378737541528239202657807309
18 Veneto	2839418	18285	155,286737763193874760732841126606508067
19 Calabria	440274	1114	395,219030520646319569120287253141831239
20 Toscana	1496786	9563	156,518456551291435741921991007006169612
21 Umbria	329835	1394	236,610473457675753228120516499282639885





VISTE

Una vista è una tabella che viene descritta in termini di altre tabelle – dette di base – o, ricorsivamente, di viste precedentemente definite.

In generale, una vista è una relazione virtuale, nel senso che le sue tuple non sono effettivamente memorizzate nella base di dati, quanto piuttosto ricavabili, attraverso interrogazioni, da altre tuple presenti nella base di dati. Una vista costituisce dunque una interfaccia da mettere a disposizione di utenti o applicazioni per le interrogazioni: si tratta di dati usati di frequente che possono anche non esistere fisicamente.

INDICI

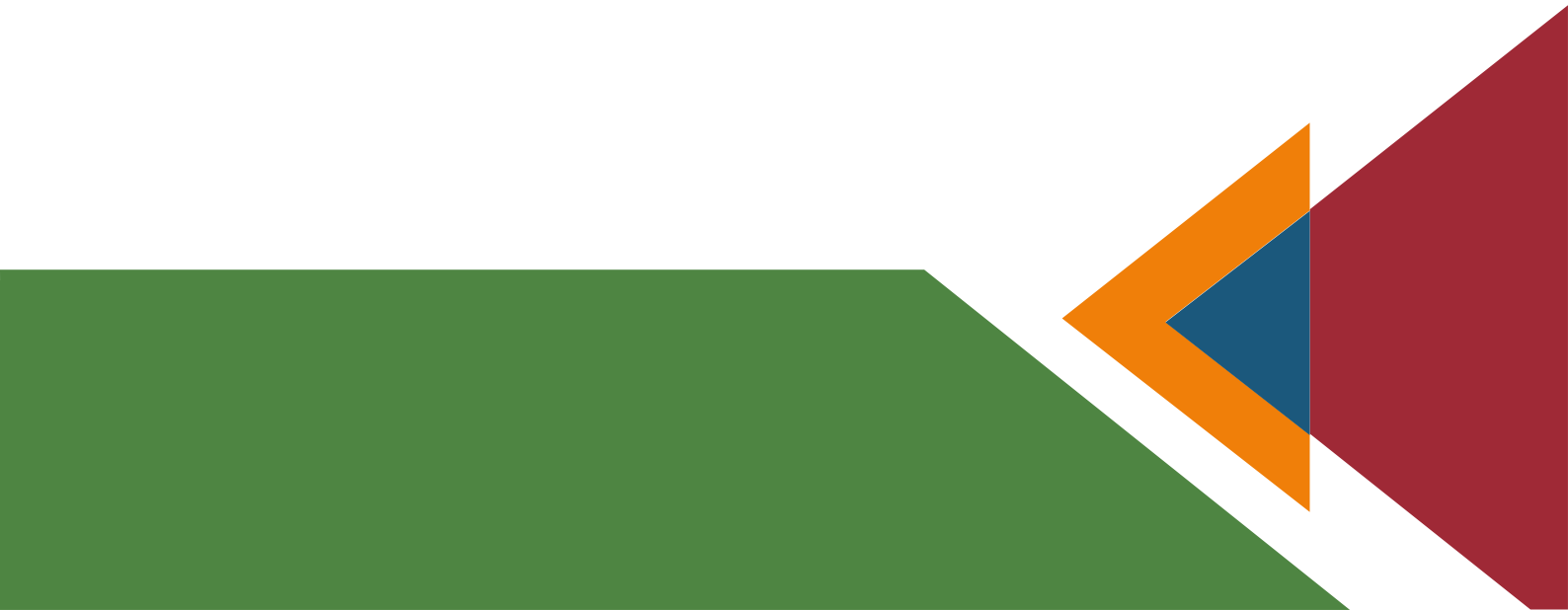
Un **indice** è una struttura dati realizzata per migliorare i tempi di ricerca (query) dei dati.

Se una tabella non ha indici, ogni ricerca obbliga il sistema a leggere tutti i dati presenti in essa. L'indice consente invece di ridurre l'insieme dei dati da leggere per completare la ricerca.

Distinguiamo gli indici primari, secondari e clustering: i primi sono composti da chiavi di ricerca che contengono attributi della chiave primaria, i restanti hanno la chiave di ricerca formata da attributi non chiave. Un file dati può avere un solo indice primario, un solo indice di clustering e diversi indici secondari. Il DBMS utilizzato (Oracle) definisce automaticamente gli indici primari sulla chiave primaria di ogni relazione, pertanto gli indici creati manualmente sono secondari.

Per dimostrare al meglio l'utilità degli Indici e delle Viste per velocizzare le query, creiamo delle semplici query per analizzare i dati dei contagi totali nelle tre sezioni delle regioni d'Italia: nord, centro e sud per analizzare la velocità di esecuzione prima senza indici e viste e poi con.

In particolare, verranno create viste che riepilogano i dati inerenti alle regioni dell'Italia Settentrionale, Centrale e Meridionale, facilitando l'esecuzione delle query che interessano quei campi. Verranno memorizzati nelle viste (materializzate) il totale casi, il numero degli ospedalizzati, ricoverati, ricoverati in terapia intensiva, numero di dimessi, in isolamento domiciliare, numero di morti e totale tamponi effettuati.



45VISTA SETTENTRIONE

DENOMINAZIONE	POSITIVI_TOTALI	OSPEDALIZZATI	RICOVERATI	RICOVERATI_IN_TERAPIA_INTENSIVA	TAMPONI_EFFETTUATI	TOTALE_CASI	ISOLAMENTO_DOMICILIARE	GUARITI	MORTI
1 Emilia-Romagna	25998	173356	157761	15595	4602032	865816	373834	208854	109772
2 Friuli Venezia Giulia	3072	9934	8263	1671	1552896	102155	46283	37708	8230
3 Liguria	8359	50004	43639	6365	1099564	237518	87056	68083	32375
4 Lombardia	77356	580902	523778	57124	9891013	2640680	854732	754138	450908
5 P.A. Bolzano	2536	10787	9055	1732	1032544	84726	41972	24036	7931
6 P.A. Trento	4247	15833	13388	2445	816925	125198	58682	38876	11807
7 Piemonte	27427	161703	144618	17085	3310065	727223	356745	132383	76392
8 Valle d'Aosta	1142	4977	4278	699	168095	38291	15452	13717	4145
9 Veneto	18285	78151	65897	12254	9249484	597351	332090	149358	37752



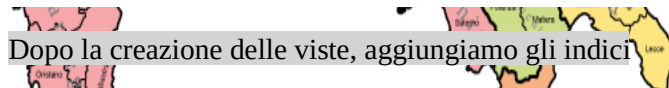
VISTA CENTRO

DENOMINAZIONE	POSITIVI_TOTALI	OSPEDALIZZATI	RICOVERATI	RICOVERATI_IN_TERAPIA_INTENSIVA	TAMPONI_EFFETTUATI	TOTALE_CASI	ISOLAMENTO_DOMICILIARE	GUARITI	MORTI
1 Lazio	6806	63731	56312	7419	3282527	204271	93323	35080	12137
2 Marche	6319	48464	42227	6237	1315196	227303	103677	46174	28988
3 Toscana	9563	53494	42651	10843	3441254	301787	182228	44777	21288
4 Umbria	1394	7354	5729	1625	843535	54095	20245	24400	2096



VISTA MERIDIONE

DENOMINAZIONE	POSITIVI_TOTALI	OSPEDALIZZATI	RICOVERATI	RICOVERATI_IN_TERAPIA_INTENSIVA	TAMPONI_EFFETTUATI	TOTALE_CASI	ISOLAMENTO_DOMICILIARE	GUARITI	MORTI
1 Abruzzo	2996	17090	14690	2400	908951	90303	53146	11380	8687
2 Basilicata	386	2606	2075	531	238246	12580	7088	2206	680
3 Calabria	1114	6851	6227	624	839400	37670	23930	4383	2506
4 Campania	4484	27734	23777	3957	1676997	143897	83600	22349	10214
5 Molise	301	1451	1200	251	129779	9953	5989	1868	645
6 Puglia	4144	27769	24418	3351	1450267	124032	72299	13178	10786
7 Sardegna	1319	5771	4791	980	533709	44071	27433	7939	2928
8 Sicilia	3240	23790	21245	2545	1725619	100187	57254	12754	6389



Dopo la creazione delle viste, aggiungiamo gli indici

```
CREATE INDEX REGIONI_NORD ON SETTENTRIONE(denominazione);
CREATE INDEX REGIONI_SUD ON MERIDIONE(denominazione);
CREATE INDEX REGIONI_CENTRO ON CENTRO(denominazione);
```



ANALISI REGIONI MERIDIONALI

CASI TOTALI PER OGNI REGIONE MERIDIONALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 8 IN 0,009 SECONDI

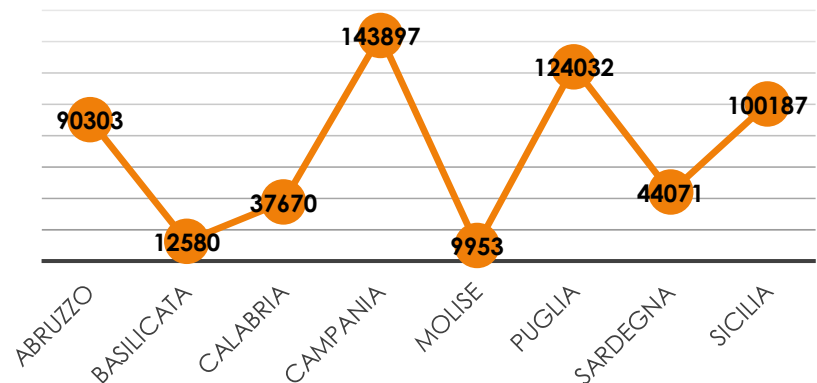
```
SELECT R.denominazione_regione, SUM(C.Totale_Casi) AS CASI_TOTALI
FROM REGIONI R JOIN COVID_REGIONI C ON R.Codice_regione=C.Codice_regione
WHERE R.DENOMINAZIONE_REGIONE = 'Sardegna' OR R.DENOMINAZIONE_REGIONE = 'Sicilia' OR R.DENOMINAZIONE_REGIONE = 'Abruzzo'
OR R.DENOMINAZIONE_REGIONE = 'Molise' OR R.DENOMINAZIONE_REGIONE = 'Campania' OR R.DENOMINAZIONE_REGIONE = 'Puglia' OR
R.DENOMINAZIONE_REGIONE = 'Basilicata' OR R.DENOMINAZIONE_REGIONE = 'Calabria'
GROUP BY R.Denominazione_regione
ORDER BY R.Denominazione_regione;
```

DENOMINAZIONE_REGIONE	CASI_TOTALI
1 Abruzzo	90303
2 Basilicata	12580
3 Calabria	37670
4 Campania	143897
5 Molise	9953
6 Puglia	124032
7 Sardegna	44071
8 Sicilia	100187

CON VISTE E CON INDICI

TUTTE LE RIGHE RECUPERATE: 8 IN 0,003 SECONDI

DENOMINAZIONE	TOTALE_CASI
1 Abruzzo	90303
2 Basilicata	12580
3 Calabria	37670
4 Campania	143897
5 Molise	9953
6 Puglia	124032
7 Sardegna	44071
8 Sicilia	100187



TOTALE OSPEDALIZZATI, DIMESSI, IN ISOLAMENTO DOMICILIARE E MORTI PER REGIONE MERIDIONALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE IN: 8 IN 0,011 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE, SUM(C.TOTALE_OSPEDALIZZATI) AS OSPEDALIZZATI, SUM(C.DIMESSI_GUARITI) AS GUARITI, SUM(C.ISOLAMENTO_DOMICILIARE) AS IN_ISOLAMENTO, SUM(C.DECEDUTI) AS MORTI
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Sardegna' OR R.DENOMINAZIONE_REGIONE = 'Sicilia' OR R.DENOMINAZIONE_REGIONE = 'Abruzzo'
OR R.DENOMINAZIONE_REGIONE = 'Molise' OR R.DENOMINAZIONE_REGIONE = 'Campania' OR R.DENOMINAZIONE_REGIONE = 'Puglia' OR
R.DENOMINAZIONE_REGIONE = 'Basilicata' OR R.DENOMINAZIONE_REGIONE = 'Calabria'
GROUP BY R.denominazione_regione
ORDER BY R.denominazione_regione;
```

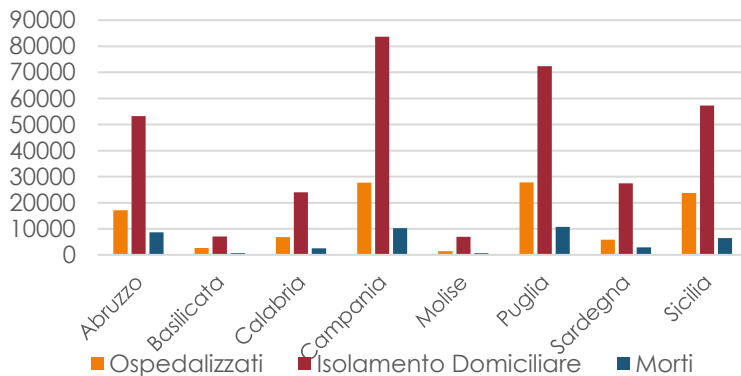
DENOMINAZIONE_REGIONE	OSPEDALIZZATI	GUARITI	IN_ISOLAMENTO	MORTI
1 Abruzzo	17090	11380	53146	8687
2 Basilicata	2606	2206	7088	680
3 Calabria	6851	4383	23930	2506
4 Campania	27734	22345	83600	10214
5 Molise	1451	1868	5989	645
6 Puglia	27769	13178	72299	10786
7 Sardegna	5771	7639	27433	2928
8 Sicilia	23790	12754	57254	6389

CON VISTE E CON INDICI

TUTTE LE RIGHE RECUPERATE: 8 IN 0,001 SECONDI

```
SELECT DENOMINAZIONE, OSPEDALIZZATI, ISOLAMENTO_DOMICILIARE, MORTI
FROM R_MERIDIONALI
ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	OSPEDALIZZATI	ISOLAMENTO_DOMICILIARE	MORTI
1 Abruzzo	17090	53146	8687
2 Basilicata	2606	7088	680
3 Calabria	6851	23930	2506
4 Campania	27734	83600	10214
5 Molise	1451	5989	645
6 Puglia	27769	72299	10786
7 Sardegna	5771	27433	2928
8 Sicilia	23790	57254	6389



TOTALE RICOVERATI CON SINTOMI E IN TERAPIA INTENSIVA PER OGNI REGIONE MERIDIONALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 8 IN 0,005 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE, SUM(C.RICOVERATI_CON_SINTOMI), SUM(C.TERAPIA_INTENSIVA)
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Sardegna' OR R.DENOMINAZIONE_REGIONE = 'Sicilia' OR R.DENOMINAZIONE_REGIONE = 'Abruzzo'
OR R.DENOMINAZIONE_REGIONE = 'Molise' OR R.DENOMINAZIONE_REGIONE = 'Campania' OR R.DENOMINAZIONE_REGIONE = 'Puglia' OR
R.DENOMINAZIONE_REGIONE = 'Basilicata' OR R.DENOMINAZIONE_REGIONE = 'Calabria'
GROUP BY R.denominazione_regione
ORDER BY R.denominazione_regione;
```

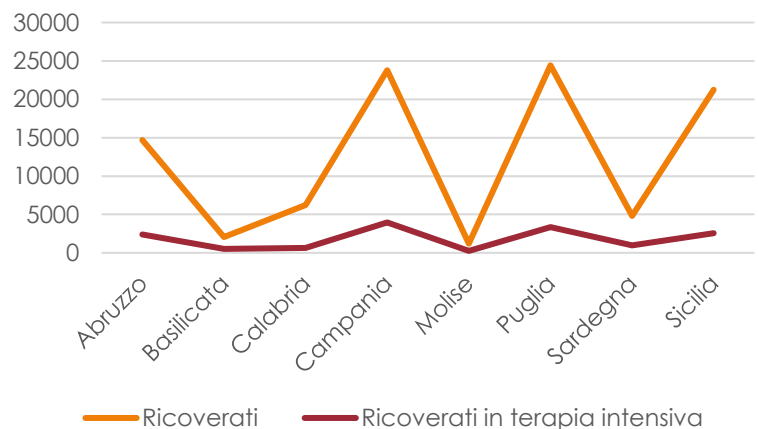
DENOMINAZIONE_REGIONE	SUM(C.RICOVERATI_CON_SINTOMI)	SUM(C.TERAPIA_INTENSIVA)
1 Abruzzo	14690	2400
2 Basilicata	2075	531
3 Calabria	6227	624
4 Campania	23777	3957
5 Molise	1200	251
6 Puglia	24418	3351
7 Sardegna	4791	980
8 Sicilia	21245	2545

CON VISTE E INDICI

TUTTE LE RIGHE RECUPERATE: 8 IN 0,007 SECONDI

```
SELECT DENOMINAZIONE, RICOVERATI, RICOVERATI_IN_TERAPIA_INTENSIVA
FROM R_MERIDIONALI
ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	RICOVERATI	RICOVERATI_IN_TERAPIA_INTENSIVA
1 Abruzzo	14690	2400
2 Basilicata	2075	531
3 Calabria	6227	624
4 Campania	23777	3957
5 Molise	1200	251
6 Puglia	24418	3351
7 Sardegna	4791	980
8 Sicilia	21245	2545



PERCENTUALE POPOLAZIONE GUARITA E CONTAGIATA SU TOTALE CASI PER OGNI REGIONE MERIDIONALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 8 IN 0,016 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE, R.NUMERO_ABITANTI, SUM(C.DIMESSI_GUARITI) AS GUARITI, SUM(C.NUOVI_POSITIVI) AS POSITIVI_TOTALI, (SUM(C.NUOVI_POSITIVI)/R.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_CONTAGIATA, (SUM(C.DIMESSI_GUARITI)/R.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_GUARITA
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Sardegna' OR R.DENOMINAZIONE_REGIONE = 'Sicilia' OR R.DENOMINAZIONE_REGIONE = 'Abruzzo'
OR R.DENOMINAZIONE_REGIONE = 'Molise' OR R.DENOMINAZIONE_REGIONE = 'Campania' OR R.DENOMINAZIONE_REGIONE = 'Puglia' OR
R.DENOMINAZIONE_REGIONE = 'Basilicata' OR R.DENOMINAZIONE_REGIONE = 'Calabria'
GROUP BY R.denominazione_regione, R.numero_abitanti
ORDER BY R.denominazione_regione;
```

DENOMINAZIONE_REGIONE	NUMERO_ABITANTI	GUARITI	POSITIVI_TOTALI	PERCENTUALE_POPOLAZIONE_CONTAGIATA	PERCENTUALE_POPOLAZIONE_GUARITA
1 Abruzzo	1305770	11380	2596	0,2294431638037326634859125267083789641361	0,8715164231598390221861430420364995366718
2 Basilicata	556934	2206	386	0,069308032908746817366793690742529635468	0,3960972036183820704069063838803161595449
3 Calabria	1924701	4383	1114	0,0578791199256403981709366805543380814994	0,22772368279540562404238372609563771120394
4 Campania	5785861	22349	4484	0,077499269339515760921321822283670289867	0,3862692173213286665545542832778042887653
5 Molise	302265	1868	301	0,099581493060724806355350437529981969464	0,618000760921707772900662663556812730551
6 Puglia	4008296	13178	4144	0,103385578310503845069728782505084454841	0,3287681348882417865347269762512548973429
7 Sardegna	1630474	7939	1319	0,080896720830874948021250262193693676955	0,486913621437692352039958932187817756089
8 Sicilia	4968410	12754	3240	0,0652120094758685374194158694632689331194	0,2567018422392676932861820984983123373474

CON INDICI E CON VISTE

TUTTE LE RIGHE RECUPERATE: 8 IN 0,006 SECONDI

```
SELECT R.DENOMINAZIONE, C.NUMERO_ABITANTI, R.GUARITI, R.POSITIVI_TOTALI,
(R.POSITIVI_TOTALI/C.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_CONTAGIATA,
(R.GUARITI/C.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_GUARITA
FROM MERIDIONE R JOIN REGIONI C ON R.DENOMINAZIONE = C.DENOMINAZIONE_REGIONE
ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	NUMERO_ABITANTI	GUARITI	POSITIVI_TOTALI	PERCENT...	PERCENTUALE...
1 Abruzzo	1305770	11380	2596	0,22944...	0,871516423...
2 Basilicata	556934	2206	386	0,06930...	0,396097203...
3 Calabria	1924701	4383	1114	0,05787...	0,227723682...
4 Campania	5785861	22349	4484	0,07749...	0,386269217...
5 Molise	302265	1868	301	0,09958...	0,618000760...
6 Puglia	4008296	13178	4144	0,10338...	0,328768134...
7 Sardegna	1630474	7939	1319	0,08089...	0,486913621...
8 Sicilia	4968410	12754	3240	0,06521...	0,256701842...

REGIONE MERIDIONALE CON MINOR NUMERO DI PERSONE IN ISOLAMENTO DOMICILIARE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 1 IN 0,018 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE AS REGIONE, SUM(C.ISOLAMENTO_DOMICILIARE)
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Sardegna' OR R.DENOMINAZIONE_REGIONE = 'Sicilia' OR R.DENOMINAZIONE_REGIONE = 'Abruzzo'
OR R.DENOMINAZIONE_REGIONE = 'Molise' OR R.DENOMINAZIONE_REGIONE = 'Campania' OR R.DENOMINAZIONE_REGIONE = 'Puglia' OR
R.DENOMINAZIONE_REGIONE = 'Basilicata' OR R.DENOMINAZIONE_REGIONE = 'Calabria'
GROUP BY R.DENOMINAZIONE_REGIONE
HAVING SUM (C.ISOLAMENTO_DOMICILIARE) <= ALL ( SELECT SUM(C.ISOLAMENTO_DOMICILIARE)
FROM REGIONI R JOIN COVID_REGIONI C ON R.CODICE_REGIONE = C.CODICE_REGIONE IE
GROUP BY R.DENOMINAZIONE_REGIONE
);
```

REGIONE	SUM(C.ISOLAMENTO_DOMICILIARE)
1 Molise	5989

CON INDICI E VISTE

TUTTE LE RIGHE RECUPERATE: 1 IN 0,006 SECONDI

```
SELECT DENOMINAZIONE, ISOLAMENTO_DOMICILIARE
FROM MERIDIONE
GROUP BY DENOMINAZIONE, ISOLAMENTO_DOMICILIARE
HAVING ISOLAMENTO_DOMICILIARE <= ALL ( SELECT ISOLAMENTO_DOMICILIARE
FROM MERIDIONE );
```

DENOMINAZIONE	ISOLAMENTO_DOMICILIARE
1 Molise	5989



ANALISI REGIONI CENTRALI

CASI TOTALI PER OGNI REGIONE CENTRALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 4 IN 0,004 SECONDI

```
SELECT R.denominazione_regione, SUM(C.Totale_Casi) AS CASI_TOTALI
FROM REGIONI R JOIN COVID_REGIONI C ON R.Codice_regione=C.Codice_regione
WHERE R.DENOMINAZIONE_REGIONE = 'Toscana' OR R.DENOMINAZIONE_REGIONE = 'Umbria' OR R.DENOMINAZIONE_REGIONE = 'Lazio' OR R.DENOMINAZIONE_REGIONE = 'Marche'
GROUP BY R.Denominazione_regione
ORDER BY R.Denominazione_regione;
```

risultato query x

Tutte le righe recuperate: 4 in 0,004 secondi

DENOMINAZIONE_REGIONE	CASI_TOTALI
1 Lazio	204271
2 Marche	227303
3 Toscana	301787
4 Umbria	54095

CON VISTE E CON INDICI

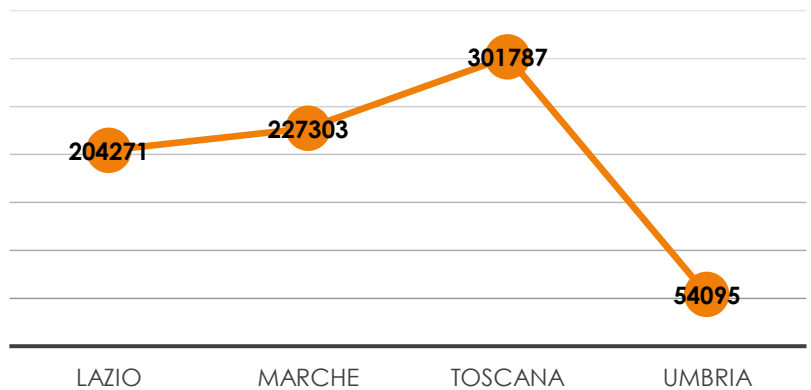
TUTTE LE RIGHE RECUPERATE: 4 IN 0,001 SECONDI

```
SELECT DENOMINAZIONE, TOTALE_CASI
FROM CENTRO
ORDER BY DENOMINAZIONE;
```

Output script x Risultato query x

Tutte le righe recuperate: 4 in 0,001 secondi

DENOMINAZIONE	TOTALE_CASI
1 Lazio	204271
2 Marche	227303
3 Toscana	301787
4 Umbria	54095



TOTALE OSPEDALIZZATI, DIMESSI, IN ISOLAMENTO DOMICILIARE E MORTI PER REGIONE CENTRALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE IN: 4 IN 0,009 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE, SUM(C.TOTALE_OSPEDALIZZATI) AS OSPEDALIZZATI, SUM(C.DIMESSI_GUARITI) AS GUARITI, SUM(C.ISOLAMENTO_DOMICILIARE) AS IN_ISOLAMENTO, SUM(C.DECEDUTI) AS MORTI
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Toscana' OR R.DENOMINAZIONE_REGIONE = 'Umbria' OR R.DENOMINAZIONE_REGIONE = 'Lazio' OR R.DENOMINAZIONE_REGIONE = 'Marche'
GROUP BY R.denominazione_regione
ORDER BY R.denominazione_regione;
```

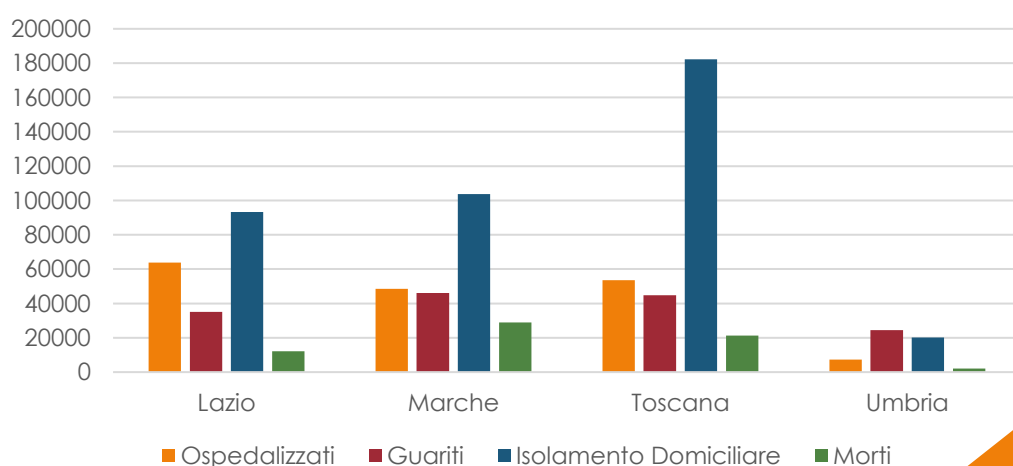
DENOMINAZIONE_REGIONE	OSPEDALIZZATI	GUARITI	IN_ISOLAMENTO	MORTI
1 Lazio	63731	35080	93323	12137
2 Marche	48464	46174	103677	28988
3 Toscana	53494	44777	182228	21288
4 Umbria	7354	24400	20245	2096

CON INDICI E CON VISTE

TUTTE LE RIGHE RECUPERATE: 4 IN 0,003 SECONDI

```
SELECT DENOMINAZIONE, OSPEDALIZZATI, GUARITI, ISOLAMENTO_DOMICILIARE, MORTI
FROM CENTRO
ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	OSPEDALIZZATI	GUARITI	ISOLAMENTO_DOMICILIARE	MORTI
1 Lazio	63731	35080	93323	12137
2 Marche	48464	46174	103677	28988
3 Toscana	53494	44777	182228	21288
4 Umbria	7354	24400	20245	2096



TOTALE RICOVERATI CON SINTOMI E IN TERAPIA INTENSIVA PER OGNI REGIONE MERIDIONALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 4 in 0,005 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE, SUM(C.RICOVERATI_CON_SINTOMI), SUM(C.TERAPIA_INTENSIVA)
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Toscana' OR R.DENOMINAZIONE_REGIONE = 'Umbria' OR R.DENOMINAZIONE_REGIONE = 'Lazio' OR R.DENOMINAZIONE_REGIONE = 'Marche'
GROUP BY R.denominazione_regione
ORDER BY R.denominazione_regione;
```

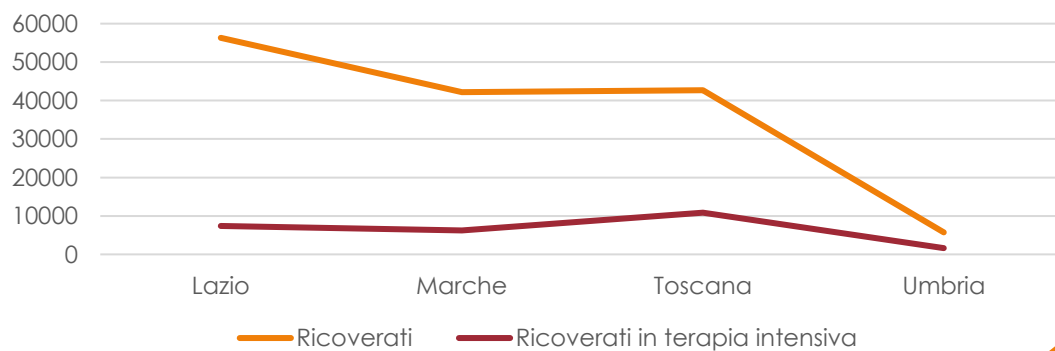
DENOMINAZIONE_REGIONE	SUM(C.RICOVERATI_CON_SINTOMI)	SUM(C.TERAPIA_INTENSIVA)
1 Lazio	56312	7419
2 Marche	42227	6237
3 Toscana	42651	10843
4 Umbria	5729	1625

CON VISTE E CON INDICI

TUTTE LE RIGHE RECUPERATE: 4 IN 0,002 SECONDI

```
SELECT DENOMINAZIONE, RICOVERATI, RICOVERATI_IN_TERAPIA_INTENSIVA
FROM CENTRO
ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	RICOVERATI	RICOVERATI_IN_TERAPIA_INTENSIVA
1 Lazio	56312	7419
2 Marche	42227	6237
3 Toscana	42651	10843
4 Umbria	5729	1625



PERCENTUALE POPOLAZIONE GUARITA E CONTAGIATA SU TOTALE CASI PER OGNI REGIONE CENTRALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 4 IN 0,005 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE, R.NUMERO_ABITANTI, SUM(C.DIMESSI_GUARITI) AS GUARITI, SUM(C.NUOVI_POSITIVI) AS POSITIVI_TOTALI, (SUM(C.NUOVI_POSITIVI)/R.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_CONTAGIATA, (SUM(C.DIMESSI_GUARITI)/R.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_GUARITA FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE WHERE R.DENOMINAZIONE_REGIONE = 'Toscana' OR R.DENOMINAZIONE_REGIONE = 'Umbria' OR R.DENOMINAZIONE_REGIONE = 'Lazio' OR R.DENOMINAZIONE_REGIONE = 'Marche' GROUP BY R.denominazione_regione, R.numero_abitanti ORDER BY R.denominazione_regione;
```

DENOMINAZIONE_REGIONE	NUMERO_ABITANTI	GUARITI	POSITIVI_TOTALI	PERCENTUALE_POPOLAZIONE_CONTAGIATA	PERCENTUALE_POPOLAZIONE_GUARITA
1 Lazio	5865544	35080	6806	0,116033568241922658836077267513465076726	0,5980689941120550796311475968810355080149
2 Marche	1518400	46174	6319	0,4161617492096944151738672286617492096944	3,04096417281348788198103266596417281349
3 Toscana	3722729	44777	9563	0,2568814436935914486388882994169062534501	1,20280041872508044501762013834474655555
4 Umbria	880285	24400	1394	0,1583578045746547992879546396905547635141	2,77182957792078701784081291854342627672

CON INDICI E CON VISTE

TUTTE LE RIGHE RECUPERATE: 4 IN 0,003

```
SELECT R.DENOMINAZIONE, C.NUMERO_ABITANTI, R.GUARITI, R.POSITIVI_TOTALI, (R.POSITIVI_TOTALI/C.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_CONTAGIATA, (R.GUARITI/C.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_GUARITA FROM CENTRO R JOIN REGIONI C ON R.DENOMINAZIONE = C.DENOMINAZIONE_REGIONE ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	NUMERO_ABITANTI	GUARITI	POSITIVI_TOTALI	PERCEN...	PERCE...
1 Lazio	5865544	35080	6806	0,116...	0,59806...
2 Marche	1518400	46174	6319	0,416...	3,04096...
3 Toscana	3722729	44777	9563	0,256...	1,20280...
4 Umbria	880285	24400	1394	0,158...	2,77182...

REGIONE CENTRALE CON MINOR NUMERO DI PERSONE IN ISOLAMENTO DOMICILIARE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 1 IN 0,007 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE AS REGIONE, SUM(C.ISOLAMENTO_DOMICILIARE) AS ISOLAMENTO_DOMICILIARE
FROM REGIONE R JOIN COVID_REGIONE C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Toscana' OR R.DENOMINAZIONE_REGIONE = 'Umbria' OR R.DENOMINAZIONE_REGIONE = 'Lazio' OR R.DENOMINAZIONE_REGIONE = 'Marche'
GROUP BY R.DENOMINAZIONE_REGIONE
HAVING SUM (C.ISOLAMENTO_DOMICILIARE) <= ALL ( SELECT SUM(C.ISOLAMENTO_DOMICILIARE)
FROM REGIONE R JOIN COVID_REGIONE C ON R.CODICE_REGIONE = C.CODICE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Toscana' OR R.DENOMINAZIONE_REGIONE = 'Umbria' OR R.DENOMINAZIONE_REGIONE = 'Lazio' OR R.DENOMINAZIONE_REGIONE = 'Marche'
GROUP BY R.DENOMINAZIONE_REGIONE );
```

REGIONE	ISOLAMENTO_DOMICILIARE
1 Umbria	20245

CON INDICI E VISTE

TUTTE LE RIGHE RECUPERATE: 1 IN 0,003 SECONDI

```
SELECT DENOMINAZIONE, ISOLAMENTO_DOMICILIARE
FROM CENTRO
GROUP BY DENOMINAZIONE, ISOLAMENTO_DOMICILIARE
HAVING ISOLAMENTO_DOMICILIARE <= ALL ( SELECT ISOLAMENTO_DOMICILIARE
FROM CENTRO);
```

DENOMINAZIONE	ISOLAMENTO_DOMICILIARE
1 Umbria	20245



ANALISI REGIONI SETTENTRIONALI

CASI TOTALI PER OGNI REGIONE SETTENTRIONALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 9 IN 0,005 SECONDI

```
SELECT R.denominazione_regione, SUM(C.Totale_Casi) AS CASI_TOTALI
FROM REGIONI R JOIN COVID_REGIONI C ON R.Codice_regione=C.Codice_regione
WHERE P.DENOMINAZIONE_REGIONE = 'Emilia-Romagna' OR P.DENOMINAZIONE_REGIONE = 'Friuli Venezia Giulia' OR P.DENOMINAZIONE_REGIONE = 'Liguria'
OR P.DENOMINAZIONE_REGIONE = 'Lombardia' OR P.DENOMINAZIONE_REGIONE = 'Piemonte' OR P.DENOMINAZIONE_REGIONE = 'P.A. Trento' OR
P.DENOMINAZIONE_REGIONE = 'Valle d'Aosta' OR P.DENOMINAZIONE_REGIONE = 'Veneto'
GROUP BY R.denominazione_regione
ORDER BY R.Denominazione_regione;
```

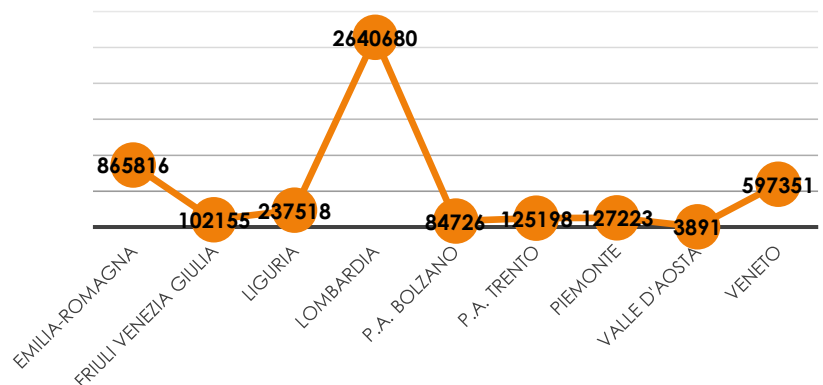
DENOMINAZIONE_REGIONE	CASI_TOTALI
1 Emilia-Romagna	865816
2 Friuli Venezia Giulia	102155
3 Liguria	237518
4 Lombardia	2640680
5 P.A. Bolzano	84726
6 P.A. Trento	125198
7 Piemonte	127223
8 Valle d'Aosta	38291
9 Veneto	597351

CON VISTE E CON INDICI

TUTTE LE RIGHE RECUPERATE: 9 IN 0,002 SECONDI

```
SELECT DENOMINAZIONE, TOTALE_CASI
FROM SETTENTRIONE
ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	TOTALE_CASI
1 Emilia-Romagna	865816
2 Friuli Venezia Giulia	102155
3 Liguria	237518
4 Lombardia	2640680
5 P.A. Bolzano	84726
6 P.A. Trento	125198
7 Piemonte	127223
8 Valle d'Aosta	38291
9 Veneto	597351



TOTALE OSPEDALIZZATI, DIMESSI, IN ISOLAMENTO DOMICILIARE E MORTI PER REGIONE SETTENTRIONALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE IN: 9 IN 0,002 SECONDI

```
=SELECT R.DENOMINAZIONE_REGIONE, SUM(C.TOTALE_OSPEDALIZZATI) AS OSPEDALIZZATI, SUM(C.DIMESSI_GUARITI) AS GUARITI, SUM(C.ISOLAMENTO_DOMICILIARE) AS IN_ISOLAMENTO, SUM(C.DECEDUTI) AS MORTI
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Emilia-Romagna' OR R.DENOMINAZIONE_REGIONE = 'Friuli Venezia Giulia' OR R.DENOMINAZIONE_REGIONE = 'Liguria'
OR R.DENOMINAZIONE_REGIONE = 'Lombardia' OR R.DENOMINAZIONE_REGIONE = 'Piemonte' OR R.DENOMINAZIONE_REGIONE = 'P.A. Trento' OR
R.DENOMINAZIONE_REGIONE = 'Valle d'Aosta' OR R.DENOMINAZIONE_REGIONE = 'Veneto'
OR R.DENOMINAZIONE_REGIONE = 'P.A. Bolzano'
GROUP BY R.denominazione_regione
ORDER BY R.denominazione_regione;
```

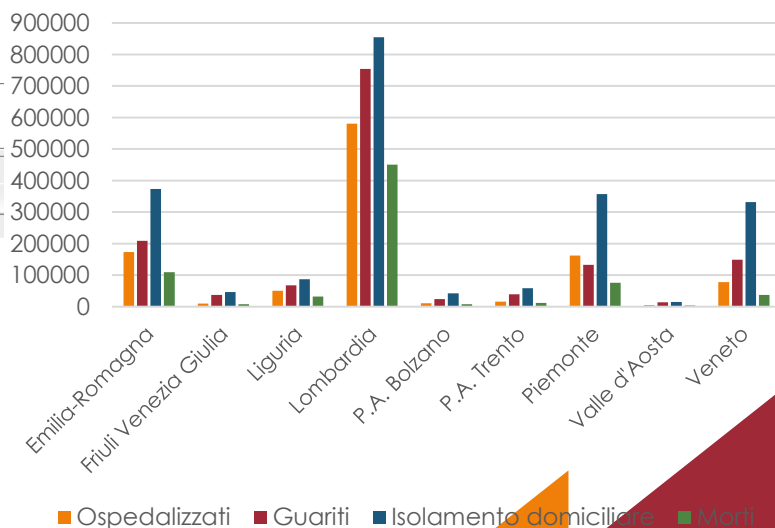
DENOMINAZIONE_REGIONE	OSPEDALIZZATI	GUARITI	IN_ISOLAMENTO	MORTI
1 Emilia-Romagna	173356	208854	373834	109772
2 Friuli Venezia Giulia	9934	37708	46283	8230
3 Liguria	50004	68083	87056	32375
4 Lombardia	580902	754138	854732	450908
5 P.A. Bolzano	10787	24036	41972	7931
6 P.A. Trento	15833	38876	58682	11807
7 Piemonte	161703	132383	356745	76392
8 Valle d'Aosta	4977	13717	15452	4145
9 Veneto	78151	149358	332090	37752

CON VISTE E CON INDICI

TUTTE LE RIGHE RECUPERATE IN: 9 IN 0,002 SECONDI

```
SELECT DENOMINAZIONE, OSPEDALIZZATI, GUARITI, ISOLAMENTO_DOMICILIARE, MORTI
FROM SETTENTRIONE
ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	OSPEDALIZZATI	GUARITI	ISOLAMENTO_DOMICILIARE	MORTI
1 Emilia-Romagna	173356	208854	373834	109772
2 Friuli Venezia Giulia	9934	37708	46283	8230
3 Liguria	50004	68083	87056	32375
4 Lombardia	580902	754138	854732	450908
5 P.A. Bolzano	10787	24036	41972	7931
6 P.A. Trento	15833	38876	58682	11807
7 Piemonte	161703	132383	356745	76392
8 Valle d'Aosta	4977	13717	15452	4145
9 Veneto	78151	149358	332090	37752



TOTALE RICOVERATI CON SINTOMI E IN TERAPIA INTENSIVA PER OGNI REGIONE SETTENTRIONALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 9 IN 0,009 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE, SUM(C.RICOVERATI_CON_SINTOMI), SUM(C.TERAPIA_INTENSIVA)
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Emilia-Romagna' OR R.DENOMINAZIONE_REGIONE = 'Friuli Venezia Giulia' OR R.DENOMINAZIONE_REGIONE = 'Liguria'
OR R.DENOMINAZIONE_REGIONE = 'Lombardia' OR R.DENOMINAZIONE_REGIONE = 'Piemonte' OR R.DENOMINAZIONE_REGIONE = 'P.A. Trento' OR
R.DENOMINAZIONE_REGIONE = 'Valle d'Aosta' OR R.DENOMINAZIONE_REGIONE = 'Veneto'
OR R.DENOMINAZIONE_REGIONE = 'P.A. Bolzano'
GROUP BY R.denominazione_regione
ORDER BY R.denominazione_regione;
```

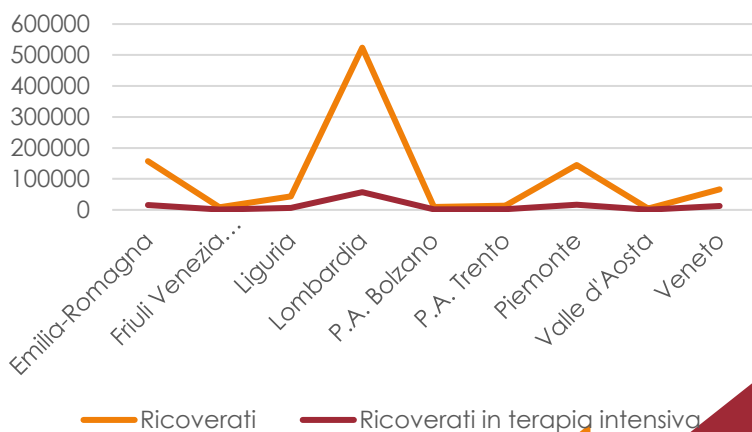
DENOMINAZIONE_REGIONE	SUM(C.RICOVERATI_CON_SINTOMI)	SUM(C.TERAPIA_INTENSIVA)
1 Emilia-Romagna	157761	15595
2 Friuli Venezia Giulia	8263	1671
3 Liguria	43639	6365
4 Lombardia	523778	57124
5 P.A. Bolzano	9055	1732
6 P.A. Trento	13388	2445
7 Piemonte	144618	17085
8 Valle d'Aosta	4278	699
9 Veneto	65897	12254

CON VISTE E CON INDICI

TUTTE LE RIGHE RECUPERATE: 9 IN 0,002 SECONDI

```
SELECT DENOMINAZIONE, RICOVERATI, RICOVERATI_IN_TERAPIA_INTENSIVA
FROM SETTENTRIONE
ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	RICOVERATI	RICOVERATI_IN_TERAPIA_INTENSIVA
1 Emilia-Romagna	157761	15595
2 Friuli Venezia Giulia	8263	1671
3 Liguria	43639	6365
4 Lombardia	523778	57124
5 P.A. Bolzano	9055	1732
6 P.A. Trento	13388	2445
7 Piemonte	144618	17085
8 Valle d'Aosta	4278	699
9 Veneto	65897	12254



PERCENTUALE POPOLAZIONE GUARITA E CONTAGIATA SU TOTALE CASI PER OGNI REGIONE SETTENTRIONALE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 9 IN 0,008 SECONDI

```
= SELECT R.DENOMINAZIONE_REGIONE, R.NUMERO_ABITANTI, SUM(C.DIMESSI_GUARITI) AS GUARITI, SUM(C.NUOVI_POSITIVI) AS POSITIVI_TOTALI, (SUM(C.NUOVI_POSITIVI)/R.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_CONTAGIATA, (SUM(C.DIMESSI_GUARITI)/R.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_GUARITA FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE WHERE R.DENOMINAZIONE_REGIONE = 'Emilia-Romagna' OR R.DENOMINAZIONE_REGIONE = 'Friuli Venezia Giulia' OR R.DENOMINAZIONE_REGIONE = 'Liguria' OR R.DENOMINAZIONE_REGIONE = 'Lombardia' OR R.DENOMINAZIONE_REGIONE = 'Piemonte' OR R.DENOMINAZIONE_REGIONE = 'P.A. Trento' OR R.DENOMINAZIONE_REGIONE = 'Valle d'Aosta' OR R.DENOMINAZIONE_REGIONE = 'P.A. Bolzano' OR R.DENOMINAZIONE_REGIONE = 'Veneto' GROUP BY R.denominazione_regione, R.numero_abitanti ORDER BY R.denominazione_regione;
```

DENOMINAZIONE_REGIONE	NUMERO_ABITANTI	GUARITI	POSITIVI_TOTALI	PERCENTUALE_POPOLAZIONE_CONTAGIATA	PERCENTUALE_POPOLAZIONE_GUARITA
1 Emilia-Romagna	4467118	208854	25998	0,581985969477412506220848219366490878459	4,675363399847507945830890553148584837025
2 Friuli Venezia Giulia	1211357	37708	3072	0,2535998883896324535211337367927043802942	3,11287258834513690018714549055315650135
3 Liguria	1543127	68083	8359	0,5416922910427981624325152758003715831555	4,41201534287197359647002482621326695729
4 Lombardia	10103969	754138	77356	0,7656001319877367003006442319844805541268	7,46377982751134727353181705129934583133
5 P.A. Bolzano	106951	24036	2536	2,37117932511149965872221858608147656403	22,4738431618217688474161064412674963301
6 P.A. Trento	117417	38876	4247	3,61702308864985479104388631969816977099	33,10934532478261239854535544256794160982
7 Piemonte	4341375	132383	27427	0,6317583714836889234401543289856324321212	3,04933344850421813365581181077423627307
8 Valle d'Aosta	125501	13717	1142	0,9099529087417630138405271671142062613047	10,92979338810049322316156843371765962024
9 Veneto	4907704	149358	18285	0,3725774822605438306792748706930980352523	3,04333757700138394654608346387638700297

CON INDICI E CON VISTE

TUTTE LE RIGHE RECUPERATE: 9 IN 0,003 SECONDI

```
= SELECT R.DENOMINAZIONE, C.NUMERO_ABITANTI, R.GUARITI, R.POSITIVI_TOTALI, (R.POSITIVI_TOTALI/C.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_CONTAGIATA, (R.GUARITI/C.NUMERO_ABITANTI)*100 AS PERCENTUALE_POPOLAZIONE_GUARITA FROM SETTENTRIONE R JOIN REGIONE C ON R.DENOMINAZIONE = C.DENOMINAZIONE_REGIONE ORDER BY DENOMINAZIONE;
```

DENOMINAZIONE	NUMERO_ABITANTI	GUARITI	POSITIVI_TOTALI	PERCENTUALE_POPOLAZIONE_CONTAGIATA	PERCENTUALE_POPOLAZIONE_GUARITA
1 Emilia-Romagna	4467118	208854	25998	0,581985969477412506220848219366490878459	4,675363399847507945830890553148584837025
2 Friuli Venezia Giulia	1211357	37708	3072	0,2535998883896324535211337367927043802942	3,11287258834513690018714549055315650135
3 Liguria	1543127	68083	8359	0,5416922910427981624325152758003715831555	4,41201534287197359647002482621326695729
4 Lombardia	10103969	754138	77356	0,7656001319877367003006442319844805541268	7,46377982751134727353181705129934583133
5 P.A. Bolzano	106951	24036	2536	2,37117932511149965872221858608147656403	22,4738431618217688474161064412674963301
6 P.A. Trento	117417	38876	4247	3,61702308864985479104388631969816977099	33,10934532478261239854535544256794160982
7 Piemonte	4341375	132383	27427	0,6317583714836889234401543289856324321212	3,04933344850421813365581181077423627307
8 Valle d'Aosta	125501	13717	1142	0,9099529087417630138405271671142062613047	10,92979338810049322316156843371765962024
9 Veneto	4907704	149358	18285	0,3725774822605438306792748706930980352523	3,04333757700138394654608346387638700297

REGIONE SETTENTRIONALE CON MINOR NUMERO DI PERSONE IN ISOLAMENTO DOMICILIARE

SENZA VISTE E SENZA INDICI

TUTTE LE RIGHE RECUPERATE: 1 IN 0,007 SECONDI

```
SELECT R.DENOMINAZIONE_REGIONE AS REGIONE, SUM(C.ISOLAMENTO_DOMICILIARE) AS ISOLAMENTO_DOMICILIARE
FROM REGIONI R JOIN COVID_REGIONI C ON R.DENOMINAZIONE_REGIONE = C.DENOMINAZIONE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Emilia-Romagna' OR R.DENOMINAZIONE_REGIONE = 'Friuli Venezia Giulia' OR R.DENOMINAZIONE_REGIONE = 'Liguria'
OR R.DENOMINAZIONE_REGIONE = 'Lombardia' OR R.DENOMINAZIONE_REGIONE = 'Piemonte' OR R.DENOMINAZIONE_REGIONE = 'P.A. Trento' OR
R.DENOMINAZIONE_REGIONE = 'Valle d'Aosta' OR R.DENOMINAZIONE_REGIONE = 'Veneto'
OR R.DENOMINAZIONE_REGIONE = 'P.A. Bolzano'
GROUP BY R.DENOMINAZIONE_REGIONE
HAVING SUM (C.ISOLAMENTO_DOMICILIARE) <= ALL ( SELECT SUM(C.ISOLAMENTO_DOMICILIARE)
FROM REGIONI R JOIN COVID_REGIONI C ON R.CODICE_REGIONE = C.CODICE_REGIONE
WHERE R.DENOMINAZIONE_REGIONE = 'Emilia-Romagna' OR R.DENOMINAZIONE_REGIONE = 'Friuli Venezia Giulia' OR R.DENOMINAZIONE_REGIONE = 'Liguria'
OR R.DENOMINAZIONE_REGIONE = 'Lombardia' OR R.DENOMINAZIONE_REGIONE = 'Piemonte' OR R.DENOMINAZIONE_REGIONE = 'P.A. Trento' OR
R.DENOMINAZIONE_REGIONE = 'Valle d'Aosta' OR R.DENOMINAZIONE_REGIONE = 'Veneto'
OR R.DENOMINAZIONE_REGIONE = 'P.A. Bolzano'
GROUP BY R.DENOMINAZIONE_REGIONE );
```

Output script x Risultato query x

Tutte le righe recuperate: 1 in 0,007 secondi

REGIONE	ISOLAMENTO_DOMICILIARE
1 Valle d'Aosta	15452

CON INDICI E VISTE

TUTTE LE RIGHE RECUPERATE: 1 IN 0,003 SECONDI

```
SELECT DENOMINAZIONE, ISOLAMENTO_DOMICILIARE
FROM SETTENTRIONE
GROUP BY DENOMINAZIONE, ISOLAMENTO_DOMICILIARE
HAVING ISOLAMENTO_DOMICILIARE <= ALL ( SELECT ISOLAMENTO_DOMICILIARE
FROM CENTRO);
```

Output script x Risultato query 3 x

Tutte le righe recuperate: 1 in 0,003 secondi

DENOMINAZIONE	ISOLAMENTO_DOMICILIARE
1 Valle d'Aosta	15452



Per creare query più specifiche si potrebbero anche creare viste e indici su regioni specifiche o su province.

Esempio vista riepilogo casi giorno per giorno province della Campania:

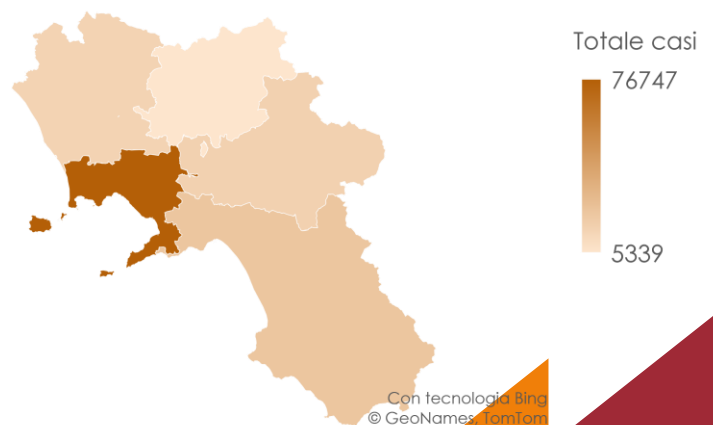
REGIONE	PROVINCIA	TOTALE_CASI	GIORNO
1	Campania Avellino	0	04-MAR-20
2	Campania Benevento	0	04-MAR-20
3	Campania Salerno	0	04-MAR-20
4	Campania Napoli	17	04-MAR-20
5	Campania Caserta	0	04-MAR-20
6	Campania Avellino	0	05-MAR-20
7	Campania Benevento	0	05-MAR-20
8	Campania Salerno	0	05-MAR-20
9	Campania Napoli	17	05-MAR-20
10	Campania Caserta	0	05-MAR-20
11	Campania Avellino	0	06-MAR-20
12	Campania Benevento	0	06-MAR-20
13	Campania Salerno	0	06-MAR-20
14	Campania Napoli	17	06-MAR-20
15	Campania Caserta	0	06-MAR-20
16	Campania Avellino	0	07-MAR-20
17	Campania Benevento	0	07-MAR-20
18	Campania Salerno	0	07-MAR-20
19	Campania Napoli	17	07-MAR-20
20	Campania Caserta	0	07-MAR-20
21	Campania Avellino	3	08-MAR-20
22	Campania Benevento	4	08-MAR-20
23	Campania Salerno	15	08-MAR-20
24	Campania Napoli	45	08-MAR-20
25	Campania Caserta	28	08-MAR-20

```
CREATE INDEX CAMPANIA ON CAMPANIA_A(provincia);
```

QUERY RIEPILOGO TOTALE CASI PER PROVINCE CAMPANE

```
SELECT PROVINCIA, SUM(TOTALE_CASI) AS CASI_TOTALI
FROM CAMPANIA_A
GROUP BY PROVINCIA
ORDER BY PROVINCIA;
```

PROVINCIA	CASI_TOTALI
1 Avellino	15798
2 Benevento	5339
3 Caserta	14782
4 Napoli	76747
5 Salerno	21893



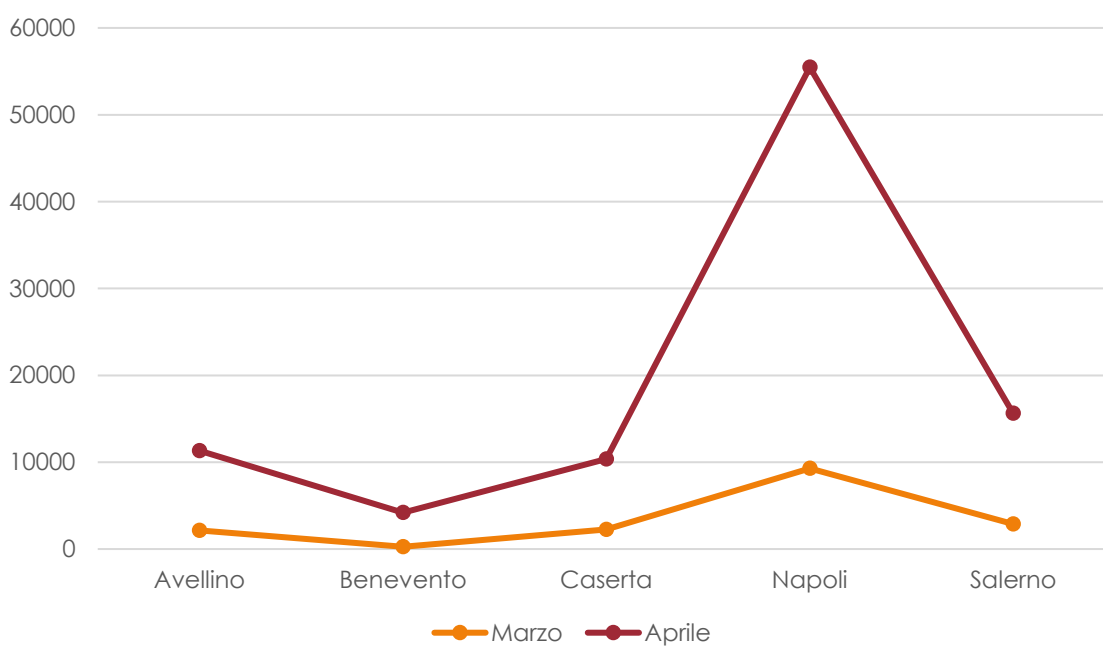
QUERY RIEPILOGO CASI TOTALI PER MESE DI MARZO E APRILE PER PROVINCE CAMPANE

```
SELECT PROVINCIA, SUM(TOTALE_CASI) AS CASI_TOTALI_MARZO
FROM CAMPANIA_A
WHERE GIORNO > '01-MAR-20' AND GIORNO < '31-MAR-20'
GROUP BY PROVINCIA
ORDER BY PROVINCIA;
```

PROVINCIA	CASI_TOTALI
1 Avellino	2155
2 Benevento	261
3 Caserta	2261
4 Napoli	9281
5 Salerno	2898

```
SELECT PROVINCIA, SUM(TOTALE_CASI) AS CASI_TOTALI_APRILE
FROM CAMPANIA_A
WHERE GIORNO > '01-APR-20' AND GIORNO < '30-APR-20'
GROUP BY PROVINCIA
ORDER BY PROVINCIA;
```

PROVINCIA	CASI_TOTALI_APRILE
1 Avellino	11314
2 Benevento	4192
3 Caserta	10378
4 Napoli	55464
5 Salerno	15647



STORED – PROCEDURE

Le procedure e funzioni in PL/SQL sono simili a quelle dei linguaggi procedurali e consistono in blocchi PL/SQL che è possibile attivare mediante una chiamata a procedura o funzione e possono essere dotate di parametri di input/output.

```
CREATE OR REPLACE PROCEDURE Conta_Contagiati_per_Regione
(
  Regione IN covid.denominazione_regione%TYPE,
  Giorno IN COVID.data%TYPE
)
AS
BEGIN
  DECLARE
    Numero_Contagiati COVID.totale_casi%TYPE;
    Cont INTEGER;
    Regione_Errata EXCEPTION;
  BEGIN
    SELECT COUNT(*) INTO Cont
    FROM COVID WHERE denominazione_regione=Regione;
    IF (Cont=0) THEN
      RAISE Regione_Errata;
    ELSE
      SELECT SUM(totale_casi) INTO Numero_Contagiati FROM COVID
      WHERE denominazione_regione=Regione AND data=Giorno
      GROUP BY denominazione_regione;
      DBMS_OUTPUT.PUT_LINE('Numero di Contagiati ' || 'in ' || Regione || ' al giorno ' || Giorno || ': ' ||
        Numero_Contagiati);
    END IF;
  EXCEPTION
    WHEN Regione_Errata THEN
      DBMS_OUTPUT.PUT_LINE('La Regione selezionata non esiste...');
    END;
  END Conta_Contagiati_per_Regione;
EXEC Conta_Contagiati_per_Regione('Piemonte','4-MAR-20');
```

Output script x

Task completato in 0,046 secondi

Procedura PL/SQL completata correttamente.
Numero di Contagiati in Piemonte al giorno 04-MAR-20: 79

Procedura PL/SQL completata correttamente.

```

CREATE OR REPLACE PROCEDURE Regione_Più_Colpita
(
    Giorno IN COVID.data%TYPE
)
AS
BEGIN
    DECLARE
        Cont INTEGER;
        Data_Errata EXCEPTION;
        Nome_regione_colpita COVID.denominazione_regione%TYPE;
    BEGIN
        SELECT COUNT(*) INTO Cont
        FROM COVID WHERE Data=Giorno;
        IF (Cont=0) THEN
            RAISE Data_Errata;
        ELSE
            SELECT distinct denominazione_regione into Nome_regione_colpita
            FROM COVID
            WHERE data=Giorno
            GROUP BY denominazione_regione
            HAVING SUM(totale_casi) >= ALL(
                SELECT SUM(totale_casi)
                FROM COVID
                WHERE Data = Giorno
                GROUP BY denominazione_regione);
            DBMS_OUTPUT.PUT_LINE( 'Regione: ' || Nome_regione_colpita|| ' ha il maggior numero di contagiati al giorno '
            || Giorno);
        END IF;
    EXCEPTION
        WHEN Data_Errata THEN
            DBMS_OUTPUT.PUT_LINE('Il giorno selezionato non esiste...');
    END;
END Regione_Più_Colpita;

EXEC Regione_Più_Colpita('06-APR-20');

```

Output script x

Task completato in 0,027 secondi

Procedure REGIONE_PIÙ_COLPITA compilato

Regione: Lombardia ha il maggior numero di contagiati al giorno 06-APR-20

Procedura PL/SQL completata correttamente.

TRIGGER

In PL/SQL i trigger sono blocchi di istruzioni che vengono attivati in maniera automatica a valle di operazioni DDL e DML sulla base di dati e sono utilizzati per preservare l'integrità dei dati o implementare particolari regole di business. Essi si basano su tre concetti portanti: l'*Evento*, che ha il compito di accendere il trigger, la *Condizione* che deve essere verificata affinché il trigger sia eseguito; ed infine l'*Azione* che viene eseguita se la condizione sul trigger acceso è soddisfatta. Poiché il paradigma di funzionamento si basa sui citati concetti, viene detto "paradigma E-C-A".

È possibile stabilire una classificazione dei trigger anche in base ad alcune caratteristiche, quali il livello di granularità e la modalità di esecuzione.

La **granularità** è un'indicazione quantitativa sul numero di elementi su cui avviene l'attivazione dell'evento. I trigger presentano due tipi di granularità: (i) *livello di tupla* – l'attivazione avviene per ogni tupla coinvolta dall'operazione indicata sull'evento che verifica la condizione, (ii) *livello di istruzione* – l'attivazione avviene una sola volta per ciascuna istruzione DML specificata nell'evento; un trigger statement level non tiene conto del numero di tuple modificate e viene avviato soltanto una volta per ciascuna istruzione.

La **modalità di esecuzione** specifica il momento in cui il trigger deve essere eseguito. Essa è normalmente immediata, ovvero, il trigger viene eseguito subito prima (*before trigger*) o subito dopo (*after trigger*) il verificarsi dell'evento all'interno della transazione che lo attiva.

L'insieme di righe di una tabella detta "target", che sono cancellate, inserite o modificate da una operazione sulle basi di dati che attiva un trigger, è detto *transition table*.

SMISTAMENTO DATI NELLE VARIE TABELLE

```
create or replace NONEDITIONABLE TRIGGER Riempimento_Tabelle
AFTER INSERT ON COVID
FOR EACH ROW
DECLARE
Cont number;
BEGIN
    SELECT COUNT(*) into cont
    FROM REGIONI where Codice_regione=:new.Codice_regione;
    IF (cont=0) THEN
        INSERT INTO REGIONI (Codice_regione, Stato, denominazione_regione) VALUES (:new.Codice_regione, :new.Stato,
        :new.denominazione_regione);
    END IF;
    SELECT COUNT(*) INTO cont
    FROM PROVINCE where Codice_provincia=:new.Codice_provincia;
    IF (cont=0)
    THEN
        INSERT INTO PROVINCE (Codice_provincia, denominazione_provincia, Sigla_provincia, Latitudine, Longitudine, Codice_regione)
        VALUES (:new.Codice_provincia, :new.denominazione_provincia, :new.Sigla_provincia, :new.Latitudine, :new.Longitudine,
        :new.Codice_regione);
    END IF;
    SELECT COUNT(*) into cont
    FROM COVID_PROVINCIA where Codice_provincia=:new.Codice_provincia AND Data=:new.Data ;
    IF (cont=0)
    THEN
        INSERT INTO COVID_PROVINCIA (Data, Codice_provincia, Totale_casi)
        VALUES (:new.Data, :new.Codice_provincia, :new.Totale_casi);
    END IF;
END;
```

BACKUP PER LA TABELLA COVID MASTER

```
CREATE OR REPLACE TRIGGER Storico_Tabella_Master
BEFORE DELETE ON COVID
FOR EACH ROW
BEGIN
    INSERT INTO BACKUP_COVID (Data, Stato, Codice_regione, denominazione_regione, Codice_provincia, denominazione_provincia,
    Sigla_provincia, Latitudine, Longitudine, Totale_casi,
    note_it, note_en)
VALUES (:old.Data, :old.Stato, :old.Codice_regione, :old.denominazione_regione, :old.Codice_provincia, :old.denominazione_provincia,
:old.Sigla_provincia, :old.Latitudine, :old.Longitudine, :old.Totale_casi,
:old.note_it, :old.note_en);
END;
```

FASCICOLO ANDAMENTO CONTAGI COMUNE DI CASORIA

In questo paragrafo andremo ad analizzare nello specifico l'andamento del contagio da virus COVID-19 nel comune di Casoria, in provincia di Napoli.

I dati sono stati recuperati dalla pagina Facebook del Sindaco di Casoria, Avv. Raffaele Bene dove ogni giorno viene pubblicato il riassunto giornaliero sui contagi nel comune.

Verranno analizzati in particolare i dati in riferimento ai mesi di Settembre, Ottobre e Novembre, riguardanti, quindi, la terza fase della diffusione del virus.

La scelta del comune è stata fatta in quanto Casoria è il mio comune di residenza e dati ISTAT affermano che esso sia al 6° posto su 550 comuni in regione per dimensione demografica e al 73° posto su 7903 comuni in ITALIA per dimensione demografica.

Creazione tabella comune:

```
CREATE TABLE CASORIA (  
  Cod_regione NUMBER(2),  
  Cod_provincia NUMBER(3),  
  Den_regione VARCHAR2(200),  
  Den_provincia VARCHAR2(200),  
  Den_comune VARCHAR2(200),  
  lat NUMBER,  
  lon NUMBER,  
  abitanti INTEGER,  
  constraint PK_CASORIA primary key (den_comune),  
  constraint FK_CASORIA_REGIONI foreign key (cod_regione) references REGIONI(codice_regione),  
  constraint FK_CASORIA_PROVINCE foreign key (cod_provincia) REFERENCES PROVINCE(codice_provincia)  
);  
  
INSERT INTO CASORIA VALUES (15,063,'Campania','Napoli','Casoria', 40.90544509887695, 14.290074348449707,76205);  
  
SELECT * FROM CASORIA;
```

Output script x Risultato query x

Tutte le righe recuperate: 1 in 0,076 secondi

	COD_REGIONE	COD_PROVINCIA	DEN_REGIONE	DEN_PROVINCIA	DEN_COMUNE	LAT	LON	ABITANTI
1	15	63	Campania	Napoli	Casoria	40,90544509887695	14,290074348449707	76205

Creazione tabella dati riguardanti il virus:

```
CREATE TABLE CASORIA_VIRUS(  
  Data DATE,  
  PROVINCIA NUMBER(3),  
  COMUNE VARCHAR2(200),  
  positivi_attuali INTEGER,  
  guariti INTEGER,  
  deceduti INTEGER,  
  isolamento_domiciliare INTEGER,  
  ricoverati_ospedale INTEGER,  
  CONSTRAINT PK_CASORIA_VIRUS PRIMARY KEY (data, comune, provincia),  
  CONSTRAINT FK_CASORIA_VIRUS_PROVINCIA FOREIGN KEY (PROVINCIA) REFERENCES PROVINCIA (CODICE_PROVINCIA),  
  CONSTRAINT FK_CASORIA_VIRUS_COMUNE FOREIGN KEY (COMUNE) REFERENCES CASORIA (DEN_COMUNE)  
);  
  
select * from casoria_virus;
```

Output script x Risultato query x

Tutte le righe recuperate: 0 in 0,01 secondi

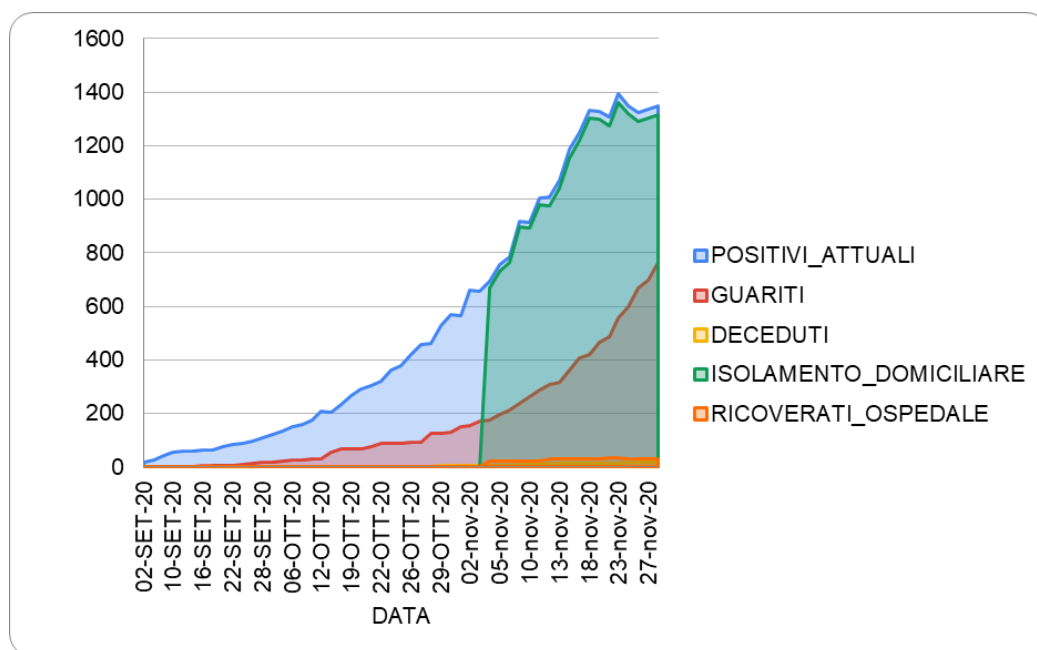
DATA	PROVINCIA	COMUNE	POSITIVI...	GUARITI	DECEDUTI	ISOLAME...	RICOVER...
------	-----------	--------	-------------	---------	----------	------------	------------

Dopo gli inserimenti ricavati dai dati forniti dal comune, avremo una tabella di questo tipo:

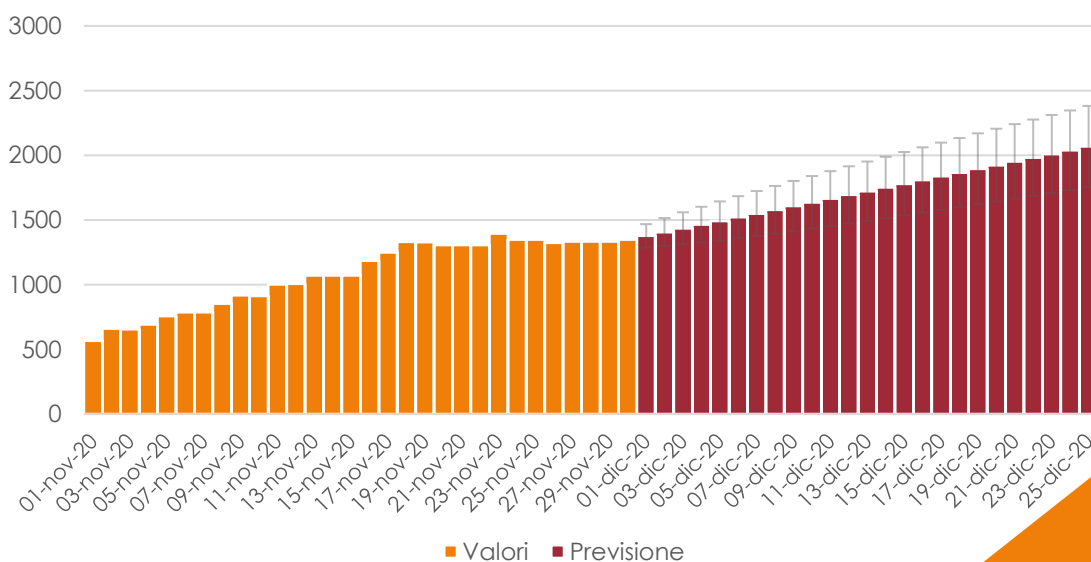
DATA	PROVINCIA	COMUNE	POSITIVI_ATTUALI	GUARITI	DECEDUTI	ISOLAMENTO_DOMICILIARE	RICOVERATI OSPEDALE
1 02-SET-20	63	Casoria	19	0	0	(null)	(null)
2 04-SET-20	63	Casoria	25	0	0	(null)	(null)
3 09-SET-20	63	Casoria	43	1	0	(null)	(null)
4 10-SET-20	63	Casoria	54	1	0	(null)	(null)
5 14-SET-20	63	Casoria	58	1	0	(null)	(null)
6 15-SET-20	63	Casoria	59	2	0	(null)	(null)
7 16-SET-20	63	Casoria	62	4	0	(null)	(null)
8 18-SET-20	63	Casoria	65	5	0	(null)	(null)
9 21-SET-20	63	Casoria	75	7	0	(null)	(null)
10 22-SET-20	63	Casoria	83	7	0	(null)	(null)
11 23-SET-20	63	Casoria	90	10	0	(null)	(null)
12 26-SET-20	63	Casoria	97	15	0	(null)	(null)
13 28-SET-20	63	Casoria	109	17	0	(null)	(null)
14 01-OTT-20	63	Casoria	120	19	0	(null)	(null)
15 02-OTT-20	63	Casoria	133	21	0	(null)	(null)
16 06-OTT-20	63	Casoria	150	25	0	(null)	(null)
17 08-OTT-20	63	Casoria	160	28	0	(null)	(null)
18 09-OTT-20	63	Casoria	175	29	0	(null)	(null)

Con i pochi dati a disposizione è possibile analizzare solo alcune caratteristiche dell'andamento del contagio nel comune:

ANDAMENTO CONTAGI SETTEMBRE – OTTOBRE – NOVEMBRE



PREVISIONI ANDAMENTO AL 25 DICEMBRE 2020



POPOLAZIONE CONTAGIATA

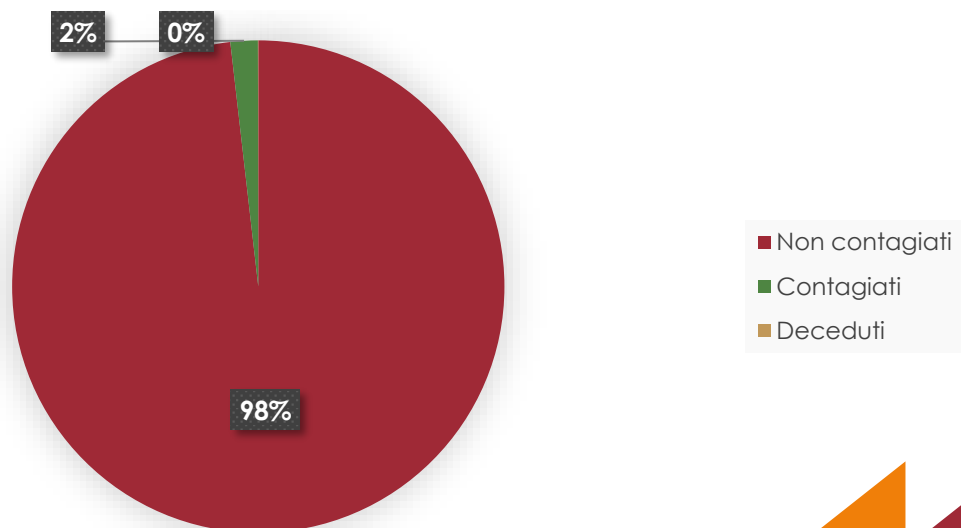
```
SELECT V.POSITIVI_ATTUALI, C.ABITANTI, (V.POSITIVI_ATTUALI * 100)/C.ABITANTI AS PERCENTUALE_CONTAGIATI
FROM CASORIA_VIRUS V JOIN CASORIA C ON V.COMUNE = C.DEN_COMUNE
WHERE V.DATA = '30-Nov-20'
GROUP BY V.POSITIVI_ATTUALI, C.ABITANTI;
```

POSITIVI_ATTUALI	ABITANTI	PERCENTUALE_CONTAGIATI
1	1349	76205 1,77022505084968177941079981628502066794

MORTALITA' VIRUS

```
SELECT POSITIVI_ATTUALI, DECEDUTI, (DECEDUTI/POSITIVI_ATTUALI) AS INDICE_MORTALITA
FROM CASORIA_VIRUS
WHERE DATA = '30-Nov-20'
GROUP BY POSITIVI_ATTUALI, DECEDUTI;
```

POSITIVI_ATTUALI	DECEDUTI	INDICE_MORTALITA
1	1349	17 0,0126019273535952557449962935507783543365

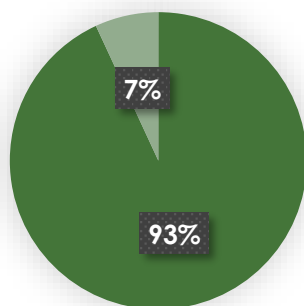


INCISIONE DATI CONTAGIO CASORIA SU PROVINCIA DI NAPOLI

Positivi totali al 30 Novembre 2020 nella provincia di Napoli: 19539

```
SELECT POSITIVI_ATTUALI, (POSITIVI_ATTUALI * 100)/19539 AS PERCENTUALE_CONTAGI_CASORIA_SU_TOTALE_PROVINCIA_DI_NAPOLI
FROM CASORIA_VIRUS
WHERE DATA = '30-Nov-20'
GROUP BY POSITIVI_ATTUALI;
```

POSITIVI_ATTUALI	PERCENTUALE_CONTAGI_CASORIA_SU_TOTALE_PROVINCIA_DI_NAPOLI
1349	6,90414043707456881109575720354163467936



- Contagi provincia di Napoli
- Contagi Casoria

PROBABILITA' DI CONTRARRE IL VIRUS AL 30 NOVEMBRE

```
SELECT POSITIVI_ATTUALI, (1/POSITIVI_ATTUALI) AS PROBABILITA_CONTAGIO
FROM CASORIA_VIRUS
WHERE DATA = '30-Nov-20'
GROUP BY POSITIVI_ATTUALI;
```

POSITIVI_ATTUALI	PROBABILITA_CONTAGIO
1349	0,00074128984432913269088213491475166790215



BIBLIOGRAFIA

1. **Chianese, Moscato, Picariello, Sansone**, *Sistemi di basi di dati e applicazioni*, Maggioli Editore, 2015.
2. **Viola Rita**, https://www.wired.it/scienza/medicina/2020/03/21/storia-coronavirus-tutte-tappe-contagio-cina-covid19/?refresh_ce=, 2020.
3. **Tuttitalia**, <https://www.tuttitalia.it/>.
4. **Wikipedia**
5. **Pagina Facebook Sindaco Raffaele Bene**, <https://www.facebook.com/Beneraffaele>

TOOL UTILIZZATI PER GRAFICI E PREVISIONI

1. Microsoft Excel
 2. Google Sheets
- 